Table des matières

Labo 11 — Contrôler la qualité 1
Avant de commencer — PyYAML 2
Étape 1 — Trois défauts qui passent au vert 3
Étape 2 — Le registre des verdicts et le module des contrats 5
Étape 3 — Le contrat des extraits de l'ERP 12
Étape 4 — La télémétrie, de la version 1.0.0 à la 2.0.0 16
Étape 5 — Lire la plateforme par ses traces 19
Étape 6 — Le contrat de l'historien 22
Étape 7 — La pression, bâtie sur la vraie sortie 25
Étape 8 — Rejouer 182 jours, et compter les alertes 29
Étape 9 — Le contrat de ce que Data Engineering livre 33
Étape 10 — Les contrats dans les DAG 36
Étape 11 — Le post-mortem du micrologiciel 2.0 39
Étape 12 — Versionner 42
Récapitulatif 43
Dépannage 44

Labo 11 — Contrôler la qualité

Labo · Cours 11/12 — Formation Data Engineering

Ce que vous allez faire. Donner à Boréal ses contrats de données : quatre fichiers YAML dans le dépôt, un module du pipeline qui traduit chaque règle en requête DuckDB, et une table où chaque verdict s'inscrit. Voir chaque contrat attraper son défaut : l'extrait livré deux fois, le mois qui n'est jamais arrivé, le champ renommé du micrologiciel 2.0, le capteur figé, la pression instable du réseau d'air. Rejouer le contrat de l'usine sur 182 jours pour compter ses alertes, puis retirer celles que personne ne lirait. Brancher les contrats dans les deux DAG, et écrire le post-mortem d'un incident de données. Objectif de fin : 22 règles jugées en quelques secondes sur tout l'historique, et un rejeu de l'usine qui passe de 107 alertes à 8, toutes vraies.

Le cours compte deux séances. La première va jusqu'à l'étape 5 et tient les contrats de ce que la `RAW` reçoit ; elle se tient seule. La seconde tient l'usine, les alertes, les DAG et l'incident.

© 2026 iAlexMG. Tous droits réservés.

Trois terminaux servent. L'Anaconda Prompt, environnement `formations` actif, lance Python et Git, depuis `Data_Domain\Projet_Boreal_intermediaire\01_de` : le pipeline, ses contrats et le rejeu. La ligne de commande DuckDB, lancée depuis `Data_Domain\Projet_Boreal_intermediaire`, lit l'entrepôt et le bronze. Ubuntu, dans WSL 2, ne sert qu'à l'étape 10, pour Airflow, depuis `Projet_Boreal_intermediaire/plateforme`. L'entrepôt n'accepte qu'un processus à la fois : quittez la ligne de commande DuckDB par `.exit` avant chaque commande du pipeline qui suit une lecture de l'entrepôt, sans quoi le pipeline s'arrête sur « L'entrepôt est occupé ».

Le journal du pipeline est montré sans la date et l'heure qui ouvrent chacune de ses lignes. Son niveau reste : dans ce cours, c'est lui qui dit ce qu'un verdict déclenche.

La liste de contrôle :

- ☐ PyYAML 6 installé dans l'environnement `formations`
- ☐ La `RAW` à jour : 204 485 messages de télémétrie, jusqu'au 2026-03-31 ; avril 2026 demandé : « à jour », 0 fichier, code 0
- ☐ La part des messages qui portent `temp_moteur_max` : 100 % en avril 2025, 0 % dès août
- ☐ `pipeline/contrats.py` écrit ; l'étape `contrats` dans la liste, hors de « tout »
- ☐ Le contrat de l'ERP : 609 commandes relivrées et 1 concessionnaire inconnu, en avertissement, code 0 ; avril 2026 : deux règles bloquantes rompues, code 1
- ☐ Le contrat de la télémétrie : 4 messages hors de la version 1.0.0 le 2025-05-01, 0 en 2.0.0
- ☐ Les traces lues : les passages de l'étape, les verdicts rompus, les 609 relivrées ramenées à leur fichier
- ☐ Le contrat de l'historien : 64 points douteux, 529 perdus aux coupures, 1 capteur figé
- ☐ La pression : 15 alertes pour la version naïve, dont 8 fausses ; 7 pour la bonne, toutes pendant la dérive
- ☐ Le rejeu de 182 jours : 107 alertes envoyées, puis 8 une fois deux règles passées en suivi
- ☐ Le contrat de l'argent : 10 heures hors de la plage de leur capteur ; les 22 règles jugées sur tout l'historique
- ☐ Les deux DAG et leur tâche de contrats : les trois mois du mensuel au vert, le 15 mars au vert, la veille au rouge sur deux règles bloquantes
- ☐ Le post-mortem écrit, et le travail commité

Avant de commencer — PyYAML

© 2026 iAlexMG. Tous droits réservés.

Les contrats s'écrivent en YAML, et le module qui les lit demande une bibliothèque de plus, PyYAML. Elle est dans le `requirements.txt` de la formation. Vérifiez qu'elle est là, dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire`, et installez-la sinon :

```
python -c "import yaml; print(yaml.__version__)"
pip install -r ..\01_Data_Engineering\requirements.txt
```

La première commande doit afficher `6.0.3`, ou une version plus récente de la série 6. Aucun service ne tourne à la première séance : les contrats lisent l'entrepôt et le lac, qui sont des fichiers. La seconde séance demandera la pile Airflow du Cours 06, à l'étape 10 seulement. Passez dans le dépôt, `cd 01_de`, pour la suite.

Étape 1 — Trois défauts qui passent au vert

Théorie : `Theorie_11_DE.md`, section 1.

Au Cours 06, vos DAG n'ont chargé que quelques journées de 2026 : le 14 janvier à la main, le 15 mars par le DAG quotidien, et les trois mois de l'ERP et des garanties par le mensuel. Remettez la `RAW` à jour d'un coup, dans l'Anaconda Prompt, depuis `01_de` :

```
python -m pipeline raw
```

```
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : toutes
INFO    pipeline : étape raw (a7bda47b) : début
INFO    pipeline :   erp_t005u                     à jour
…
INFO    pipeline :   telemetrie_sorties           88 fichier(s),   45 168 lignes
INFO    pipeline :   corpus_index                  à jour
INFO    pipeline :   RAW : 88 fichier(s) chargé(s) à ce passage
INFO    pipeline : étape raw (a7bda47b) : fin, 45 168 lignes en 1.2 s
```

Quatre-vingt-huit archives quotidiennes de télémétrie : les 90 jours de 2026, moins les deux déjà chargés. Si votre Labo 06 a pris d'autres journées, le compte diffère. Relancez la commande : elle doit dire « à jour » sur ses 22 lignes et charger 0 fichier.

Demandez maintenant à la `RAW` un mois qui n'existe pas, avril 2026, puis lisez le code de sortie :

```
python -m pipeline raw --livraisons erp,garanties --depuis 2026-04 --jusqua 2026-04
echo %ERRORLEVEL%
```

```
INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
INFO    pipeline : livraisons : erp, garanties
INFO    pipeline : étape raw (675dd7e2) : début
INFO    pipeline :   erp_t005u                     à jour
```


© 2026 iAlexMG. Tous droits réservés.

```
…
INFO    pipeline :   garanties_reclamations        à jour
INFO    pipeline :   RAW : 0 fichier(s) chargé(s) à ce passage
INFO    pipeline : étape raw (675dd7e2) : fin, 0 lignes en 0.1 s
INFO    pipeline : pipeline : terminé
0
```

Tout est « à jour », zéro ligne, code 0. Le pipeline dit vrai : il n'avait rien de nouveau à charger. Mais « rien de nouveau » et « rien du tout » s'affichent pareil, et un DAG qui aurait lancé cette commande serait passé au vert pour un mois jamais arrivé. Le chargeur ne sait pas ce qu'il devait recevoir ; il ne connaît que ce qu'il a reçu.

Ouvrez la ligne de commande DuckDB depuis `Projet_Boreal_intermediaire`, et l'entrepôt en lecture seule :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

-- Attendu : 204 485 messages, du lot 2023-09-09 au lot 2026-03-31.
SELECT COUNT(*) AS messages, MIN(_lot) AS premier_lot, MAX(_lot) AS dernier_lot
FROM raw.telemetrie_sorties;
```

messages int64	premier_lot varchar	dernier_lot varchar
204485	2023-09-09	2026-03-31

Quarante et une archives sont vides, les 39 premières, du 2023-08-01 au 2023-09-08, puis celles des 11 et 12 septembre : aucun message n'y porte leur lot. Le deuxième défaut est connu depuis le Cours 03, l'extrait de mars 2023 livré deux fois :

```
SELECT _lot, COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes
FROM raw.erp_vbak WHERE _lot LIKE '2023-03%'
GROUP BY ALL ORDER BY ALL;
```

_lot varchar	lignes int64	commandes int64
2023-03	609	609
2023-03_bis	609	609

La `RAW` garde les deux lots, et c'est voulu : le brut reste le brut. Qui additionne les commandes de 2023 sans dédoublonner en compte 609 de trop, sans la moindre erreur.

© 2026 iAlexMG. Tous droits réservés.

Le troisième se voit du côté d'un consommateur. Imaginez un tableau de bord de la température du moteur, écrit en 2024 sur le champ `temp_moteur_max`. Voici ce qu'il a lu, mois par mois, depuis mars 2025 :

```
SELECT LEFT(_lot, 7) AS mois, COUNT(*) AS messages,
       COUNT(message->>'temp_moteur_max') AS ancien_champ,
       ROUND(100.0 * COUNT(message->>'temp_moteur_max') / COUNT(*), 1) AS part,
       MAX(CAST(message->>'temp_moteur_max' AS DOUBLE)) AS temp_lue,
       MAX(CAST(COALESCE(message->>'temperature_moteur_max',
                         message->>'temp_moteur_max') AS DOUBLE)) AS temp_reelle
FROM raw.telemetrie_sorties
WHERE _lot >= '2025-03'
GROUP BY ALL ORDER BY mois;
```

mois varchar	messages int64	ancien_champ int64	part double	temp_lue double	temp_reelle double
2025-03	8493	8493	100.0	96.0	96.0
2025-04	5387	5387	100.0	102.0	102.0
2025-05	7307	5688	77.8	94.6	94.6
2025-06	10229	3611	35.3	98.0	101.7
2025-07	12673	536	4.2	93.9	101.7
2025-08	13344	0	0.0	NULL	102.0
2025-09	10203	0	0.0	NULL	102.5
2025-10	8775	0	0.0	NULL	93.9
2025-11	7386	0	0.0	NULL	97.5
2025-12	9386	0	0.0	NULL	119.6
2026-01	16076	0	0.0	NULL	117.1
2026-02	17149	0	0.0	NULL	120.7
2026-03	13114	0	0.0	NULL	119.6

`part` est la part des messages du mois qui portent encore l'ancien nom ; `temp_lue`, le maximum que ce tableau de bord affiche ; `temp_reelle`, le vrai maximum, lu sous l'un ou l'autre nom. Le micrologiciel 2.0 a renommé le champ le 2025-05-01, et la flotte a basculé en trois mois : dès août, le tableau de bord ne lit plus rien. Il n'affiche aucune erreur, seulement des cases vides, et l'hiver des surchauffes lui échappe : 119,6 °C en décembre, 120,7 °C en février.

Trois défauts, zéro erreur. Quittez DuckDB par `.exit`.

Étape 2 — Le registre des verdicts et le module des contrats

Théorie : `Theorie_11_DE.md`, sections 2 et 3.

© 2026 iAlexMG. Tous droits réservés.

Un contrat est un fichier YAML du dépôt ; une règle, une métrique, un attendu et une sévérité ; et un module du pipeline traduit chaque règle en requête. Créez le dossier des contrats et le module, dans l'Anaconda Prompt, depuis `01_de` :

```
mkdir contrats
notepad pipeline\contrats.py
```

Ce code ne se tape pas dans la ligne de commande DuckDB : c'est un fichier du dépôt. Au moment d'enregistrer, choisissez « Tous les fichiers » dans la liste des types, sans quoi le Bloc-notes ajoute `.txt` au nom. D'abord ce que le module connaît :

```
# -*- coding: utf-8 -*-
"""contrats.py : les contrats de données, lus en YAML et jugés par DuckDB (Cours 11)."""
from datetime import datetime, timezone

from .config import DEPOT
from .fenetre import Fenetre
from .journal import LOG

METRIQUES = ("lignes", "doublons", "invalides", "champs_json", "periodes_manquantes", "sql")
# Ce qu'une règle rompue déclenche : arrêter le pipeline, prévenir quelqu'un, ou seulement s'inscrire
# dans pipeline.controles pour le tableau de bord de la semaine.
SEVERITES = ("bloquant", "avertissement", "suivi")
# Un attendu, le signe qui l'affiche, et le test qu'il fait passer au nombre mesuré.
ATTENDUS = {
    "doit_etre": ("=", lambda valeur, cible: valeur == cible),
    "au_plus": ("<=", lambda valeur, cible: valeur <= cible),
    "au_moins": (">=", lambda valeur, cible: valeur >= cible),
}


class ContratRompu(RuntimeError):
    """Au moins une règle bloquante n'a pas rendu son attendu."""


class ContratIllisible(ValueError):
    """Un fichier de contrat auquel il manque ce que le module doit lire."""
```

Six métriques, dont une requête libre, `sql`. Trois sévérités. Trois attendus, chacun avec le signe qui l'affichera au journal et le test qu'il fait passer au nombre mesuré. Deux exceptions : l'une pour un contrat rompu, l'autre pour un fichier mal formé, qui doit s'arrêter avant d'envoyer la moindre requête.

Puis le registre des verdicts, et la fenêtre du Cours 06 :

```
def creer_table(con) -> None:
    """Le registre des verdicts, à côté du journal des chargements et de celui des exécutions."""
```


© 2026 iAlexMG. Tous droits réservés.

```
    con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
    con.execute("""
        CREATE TABLE IF NOT EXISTS pipeline.controles (
            controle_le TIMESTAMPTZ,     -- le même pour toutes les règles d'un passage
            fenetre     VARCHAR,         -- la tranche jugée, en toutes lettres
            contrat     VARCHAR,
            version     VARCHAR,
            regle       VARCHAR,
            dimension   VARCHAR,         -- fraicheur, volume, unicite, validite, schema...
            severite    VARCHAR,         -- bloquant, avertissement ou suivi
            valeur      DOUBLE,          -- ce que la requête a rendu
            attendu     VARCHAR,         -- ce qu'elle devait rendre, avec son signe : « = 0 »
            respectee   BOOLEAN)""")


def lots_de_la_fenetre(con, fenetre: Fenetre) -> None:
    """Les lots que la fenêtre retient, dans une table temporaire que les règles peuvent citer.

    La décision revient à Fenetre.contient, celle du Cours 06 : un lot relivré garde son mois, un
    export « complet » entre toujours. Les lots candidats sont ceux du journal des chargements.
    """
    connus = [lot for (lot,) in con.execute(
        "SELECT DISTINCT lot FROM pipeline.chargements").fetchall()]
    con.execute("CREATE OR REPLACE TEMP TABLE lots_de_la_fenetre (lot VARCHAR)")
    retenus = [(lot,) for lot in connus if fenetre.contient(lot)]
    if retenus:
        con.executemany("INSERT INTO lots_de_la_fenetre VALUES (?)", retenus)


def condition(fenetre: Fenetre, colonne: str, forme: str) -> str:
    """La condition SQL qui borne une règle à la fenêtre : sur un lot, ou sur une date."""
    if fenetre.ouverte:
        return "true"
    if forme == "lot":
        return f"{colonne} IN (SELECT lot FROM lots_de_la_fenetre)"
    bornes = []
    if fenetre.depuis:
        bornes.append(f"{colonne} >= DATE '{fenetre.depuis}'")
    if fenetre.jusqua:
        bornes.append(f"{colonne} < DATE '{fenetre.jusqua}' + 1")
    return " AND ".join(bornes)
```


© 2026 iAlexMG. Tous droits réservés.

`pipeline.controles` vit à côté de `pipeline.chargements` et de `pipeline.executions` : une ligne par règle et par passage, avec la fenêtre jugée. `lots_de_la_fenetre()` range dans une table temporaire les lots que la fenêtre retient, par `Fenetre.contient()`, la méthode du Cours 06 : un lot relivré comme `2023-03_bis` garde son mois. `condition()` rend la condition SQL qui borne une règle, sur un lot ou sur une date.

Puis la traduction des métriques :

```
  def periodes_manquantes(regle: dict, table: str, colonne: str, ou: str, fenetre: Fenetre) -> str:
      """Les jours ou les mois de la fenêtre qui n'ont rien livré, alors que la fenêtre les attend.

      Fenêtre fermée, les bornes sont les siennes : un mois d'avril demandé et absent se compte.
      Fenêtre ouverte, ce sont celles des données, et seuls les trous au milieu se comptent.
      """
      mois = regle["pas"] == "mois"
      taille, pas, forme = (7, "1 MONTH", "%Y-%m") if mois else (10, "1 DAY", "%Y-%m-%d")
      debut = "CAST(periode || '-01' AS DATE)" if mois else "CAST(periode AS DATE)"
      d = f"DATE '{fenetre.depuis}'" if fenetre.depuis else f"(SELECT MIN({debut}) FROM presentes)"
      j = f"DATE '{fenetre.jusqua}'" if fenetre.jusqua else f"(SELECT MAX({debut}) FROM presentes)"
      return f"""
          WITH presentes AS (
              SELECT DISTINCT LEFT(CAST({colonne} AS VARCHAR), {taille}) AS periode FROM {table}
              WHERE {ou} AND regexp_full_match(LEFT(CAST({colonne} AS VARCHAR), {taille}),
                                               '[0-9]{{4}}-[0-9]{{2}}(-[0-9]{{2}})?'))
          SELECT COUNT(*) FROM (SELECT UNNEST(generate_series({d}, {j}, INTERVAL {pas})) AS debut)
          WHERE strftime(debut, '{forme}') NOT IN (SELECT periode FROM presentes)"""


  def requete(regle: dict, cadre: dict, fenetre: Fenetre) -> str:
      """La requête DuckDB d'une règle : un SELECT qui rend un seul nombre."""
      colonne = regle.get("periode", cadre["colonne"])
      ou = condition(fenetre, colonne, cadre["forme"])
      if "filtre" in regle:
          ou = f"({ou}) AND ({regle['filtre']})"
      metrique, table = regle.get("metrique", "sql"), regle.get("table")
      if metrique == "sql":
          return regle["sql"].replace("{fenetre}", f"({ou})")
      if metrique == "lignes":
          return f"SELECT COUNT(*) FROM {table} WHERE {ou}"
      if metrique == "doublons":
          cles = ", ".join(regle["colonnes"])
          return (f"SELECT COALESCE(SUM(n - 1), 0) FROM "
                  f"(SELECT COUNT(*) AS n FROM {table} WHERE {ou} GROUP BY {cles})")
      if metrique == "invalides":
```


© 2026 iAlexMG. Tous droits réservés.

```
        return f"SELECT COUNT(*) FROM {table} WHERE {ou} AND NOT COALESCE({regle['valide']}, false)"
    if metrique == "champs_json":
        # Chaque version du message déclarée par le contrat est une liste de champs triée ; un
        # message dont les champs ne sont exactement aucune de ces listes est hors contrat.
        versions = ", ".join("[" + ", ".join(f"'{c}'" for c in sorted(champs)) + "]"
                             for champs in regle["versions"].values())
        return (f"SELECT COUNT(*) FROM {table} WHERE {ou} "
                f"AND list_sort(json_keys({regle['champ']})) NOT IN ({versions})")
    return periodes_manquantes(regle, table, colonne, ou, fenetre)
```

Chaque métrique devient un `SELECT` qui rend un seul nombre. `doublons` compte les lignes en trop par clé ; `invalides`, les lignes dont la condition `valide` n'est pas vraie, un `NULL` compris ; `champs_json`, les messages dont la liste triée des clés n'est aucune des versions déclarées ; `sql` remplace `{fenetre}` dans la requête du contrat. `periodes_manquantes` fabrique la liste des mois ou des jours que la fenêtre attend, par `generate_series`, et compte ceux qu'aucune ligne ne porte. C'est ce qui lui fait voir un mois absent, qu'aucune table ne peut montrer.

Le jugement d'une règle, et la lecture d'un contrat :

```
def juger(valeur: float, regle: dict) -> tuple:
    """Le verdict d'une règle, et son attendu en toutes lettres : (respectée, « = 0 »)."""
    for cle, (signe, test) in ATTENDUS.items():
        if cle in regle:
            return test(valeur, regle[cle]), f"{signe} {regle[cle]}"
    raise ContratIllisible(f"la règle {regle['nom']} n'a pas d'attendu : {', '.join(ATTENDUS)}")


def lire(fichier) -> dict:
    """Un contrat, lu et vérifié avant qu'une seule requête parte."""
    import yaml                                          # dépendance de cette étape seulement
    contrat = yaml.safe_load(fichier.read_text(encoding="utf-8"))
    manque = [cle for cle in ("contrat", "version", "fenetre", "regles") if cle not in contrat]
    if manque:
        raise ContratIllisible(f"{fichier.name} : il manque {', '.join(manque)}")
    for regle in contrat["regles"]:
        if regle.get("severite") not in SEVERITES:
            raise ContratIllisible(f"{fichier.name}, {regle.get('nom')} : sévérité "
                                   f"{', '.join(SEVERITES)}")
        if regle.get("metrique", "sql") not in METRIQUES:
            raise ContratIllisible(f"{fichier.name}, {regle['nom']} : métrique inconnue, "
                                   f"{regle['metrique']}")
    return contrat
```


© 2026 iAlexMG. Tous droits réservés.

`import yaml` est dans la fonction, et non en tête du module : PyYAML ne sert qu'à cette étape, et le chemin de « tout », celui que rejoue le kit de départ d'Analytics Engineering, continue de n'importer que DuckDB et la bibliothèque standard. `yaml.safe_load` lit le fichier sans exécuter quoi que ce soit qu'il contiendrait. `lire()` refuse un contrat sans version, ou une sévérité inconnue, avant qu'une seule requête parte.

L'exécution d'un contrat, et le choix des fichiers :

```
  def verifier(con, contrat: dict, fenetre: Fenetre) -> list:
      """Chaque règle d'un contrat, lancée et jugée. Rend une ligne par règle."""
      if contrat.get("preparation"):
          con.execute(contrat["preparation"])       # les vues que les règles partagent
      verdicts = []
      for regle in contrat["regles"]:
          valeur = con.execute(requete(regle, contrat["fenetre"], fenetre)).fetchone()[0]
          valeur = float(valeur or 0)
          respectee, attendu = juger(valeur, regle)
          verdicts.append((contrat["contrat"], str(contrat["version"]), regle["nom"],
                           regle.get("dimension"), regle["severite"], valeur, attendu, respectee))
      return verdicts


  def contrats_de(livraisons=None) -> list:
      """Les fichiers de contrat, tous ou ceux des livraisons nommées : « erp », « usine »..."""
      fichiers = sorted((DEPOT / "contrats").glob("*.yaml"))
      if not livraisons:
          return fichiers
      retenus = [f for f in fichiers if f.stem.split("_")[0] in livraisons]
      if not retenus:
          raise ContratIllisible(f"Aucun contrat pour {', '.join(livraisons)}. Connus : "
                                 f"{', '.join(f.stem for f in fichiers)}.")
      return retenus
```

`preparation` est un bloc SQL facultatif, lancé avant les règles : le contrat de l'usine y posera ses vues, à l'étape 6. `contrats_de()` choisit les fichiers par le début de leur nom : `--livraisons erp` retient `erp_extraits.yaml`, comme la même option retient les livraisons de la `RAW`.

Enfin l'étape elle-même :

```
  def executer(con, fenetre: Fenetre = None, livraisons=None) -> int:
      """Vérifie les contrats demandés. Rend le nombre de règles jugées ; lève si une bloque.

      Toutes les règles passent avant la décision : une exécution qui s'arrêterait à la première règle
      rompue cacherait les suivantes, et on corrigerait un défaut à la fois.
      """
      fenetre = fenetre or Fenetre()
```


© 2026 iAlexMG. Tous droits réservés.

```
    creer_table(con)
    lots_de_la_fenetre(con, fenetre)
    controle_le = datetime.now(timezone.utc)
    bloquantes, total = [], 0
    for fichier in contrats_de(livraisons):
        for verdict in verifier(con, lire(fichier), fenetre):
            contrat, _, regle, _, severite, valeur, attendu, respectee = verdict
            con.execute("INSERT INTO pipeline.controles VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
                        [controle_le, str(fenetre), *verdict])
            total += 1
            texte = f"{valeur:,.0f}".replace(",", " ") if valeur == int(valeur) else f"{valeur:g}"
            ligne = f"  %-10s %-52s %9s   (attendu %s)"
            if respectee:
                LOG.info(ligne, "respectée", f"{contrat}.{regle}", texte, attendu)
            elif severite == "suivi":
                LOG.info(ligne, "hors cible", f"{contrat}.{regle}", texte, attendu)
            elif severite == "avertissement":
                LOG.warning(ligne, "à voir", f"{contrat}.{regle}", texte, attendu)
            else:
                LOG.error(ligne, "ROMPUE", f"{contrat}.{regle}", texte, attendu)
                bloquantes.append(f"{contrat}.{regle}")
    if bloquantes:
        quoi = "règle bloquante rompue" if len(bloquantes) == 1 else "règles bloquantes rompues"
        raise ContratRompu(f"{len(bloquantes)} {quoi} : {', '.join(bloquantes)}")
    return total
```

Toutes les règles passent avant la décision, et chacune s'inscrit dans la table. Une règle respectée s'écrit au niveau `INFO`, comme une règle de suivi rompue, marquée « hors cible » ; une règle d'avertissement rompue au niveau `WARNING`, « à voir » ; une règle bloquante rompue au niveau `ERROR`, « ROMPUE ». Si une seule bloquante est tombée, l'étape lève `ContratRompu` à la fin : la porte d'entrée du Cours 05 l'inscrit « échoué » dans `pipeline.executions` et rend le code 1.

Branchez l'étape dans la porte d'entrée. Ouvrez `pipeline\__main__.py` et changez trois choses. La ligne d'import prend le module :

```
from . import bronze, contrats, corpus, erp, meteo, raw
```

Le dictionnaire `ETAPES` gagne une ligne, sous celle de `meteo` :

```
    "contrats": (contrats.executer, "les contrats de données, jugés sur ce qui est arrivé", False,
                 True),
```

Le troisième champ, `False`, la tient hors de « tout » ; le quatrième, `True`, lui laisse la fenêtre. Et le paragraphe de la docstring qui disait deux étapes hors de « tout » en dit maintenant trois ; remplacez-le par celui-ci :

© 2026 iAlexMG. Tous droits réservés.

```
Trois étapes ne sont pas dans « tout » : erp, qui demande le conteneur de la base et le pilote
psqlODBC ; meteo, qui demande le réseau ; et contrats (Cours 11), qui juge ce que les autres ont
écrit et demande PyYAML. Un pipeline rejouable hors ligne, avec DuckDB et la bibliothèque standard
pour seules dépendances, ne les suppose pas.
```

Listez les étapes, puis lancez les contrats, alors que le dossier n'en contient encore aucun :

```
python -m pipeline --etapes
python -m pipeline contrats
```

```
Étapes du pipeline :
  bronze   les CSV et le JSON de l'usine, rangés en Parquet   (reconstruit tout)
  raw      les 22 livraisons chargées dans la couche RAW   (fenêtre)
  corpus   la documentation déposée telle quelle dans le bronze   (reconstruit tout)
  erp      les neuf tables de la base de l'ERP, par ODBC   (hors de « tout », reconstruit tout)
  meteo    les observations d'ECCC, par son API   (hors de « tout », reconstruit tout)
  contrats les contrats de données, jugés sur ce qui est arrivé   (hors de « tout », fenêtre)
INFO    pipeline : pipeline : contrats, depuis C:\…\Data_Domain\Projet_Boreal_intermediaire
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : toutes
INFO    pipeline : étape contrats (426e3342) : début
INFO    pipeline : étape contrats (426e3342) : fin, 0 lignes en 1.3 s
INFO    pipeline : pipeline : terminé
```

Aucune règle, et pourtant un passage : la table `pipeline.controles` existe, vide, et l'étape s'est inscrite dans `pipeline.executions`. Le registre est prêt.

Étape 3 — Le contrat des extraits de l'ERP

Théorie : `Theorie_11_DE.md`, sections 3 et 4.

Premier contrat : ce que l'équipe de l'ERP livre chaque mois, tel que la `RAW` le garde. `notepad contrats\erp_extraits.yaml`, depuis `01_de`. L'en-tête, puis les deux premières règles :

```yaml
# erp_extraits.yaml : le contrat des extraits mensuels de l'ERP, tels que la RAW les garde.

contrat: erp_extraits
version: 1.0.0
produit: les extraits mensuels de l'ERP, dans les tables raw.erp_*
producteur: l'équipe de l'ERP, qui dépose un extrait par mois
consommateurs: Analytics Engineering, par le STAGING des commandes et des concessionnaires
# La colonne qui situe une ligne dans le temps, et sa forme : un lot, ou une date.
fenetre:
  colonne: _lot
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
    forme: lot

  regles:
    - nom: extrait_du_mois_livre
      dimension: fraicheur
      severite: bloquant
      metrique: periodes_manquantes
      table: raw.erp_vbak
      pas: mois
      doit_etre: 0

    - nom: commandes_du_mois
      dimension: volume
      severite: bloquant
      # Le plus petit extrait livré depuis 2022 compte 380 commandes : moins de 300, c'est un extrait
      # tronqué, pas un mois tranquille.
      sql: |
        SELECT COALESCE(MIN(n), 0) FROM (
          SELECT COUNT(*) AS n FROM raw.erp_vbak WHERE {fenetre} GROUP BY _lot)
      au_moins: 300
```

Le YAML se lit par son indentation, en espaces, jamais en tabulations : chaque règle commence par un tiret, et ses clés sont alignées sous son nom. Le `|` ouvre un bloc de texte gardé avec ses retours à la ligne, la forme de toutes les requêtes libres. `fenetre` dit comment situer une ligne dans le temps : ici par son lot, `_lot`. La première règle est une règle de fraîcheur : chaque mois de la fenêtre doit avoir un extrait. La seconde, de volume : le plus petit extrait jamais livré, celui de décembre 2024, compte 380 commandes, et un extrait de moins de 300 est un extrait tronqué, pas un mois tranquille.

Les quatre suivantes :

```yaml
    - nom: commande_unique_dans_son_lot
      dimension: unicite
      severite: bloquant
      metrique: doublons
      table: raw.erp_vbak
      colonnes: [VBELN, _lot]
      doit_etre: 0

    - nom: numeros_au_format_sap
      dimension: validite
      severite: bloquant
      metrique: invalides
      table: raw.erp_vbak
      valide: >-
```


© 2026 iAlexMG. Tous droits réservés.

```
      regexp_full_match(VBELN, '[0-9]{10}') AND regexp_full_match(KUNNR, '[0-9]{10}')
      AND regexp_full_match(ERDAT, '[0-9]{8}')
    doit_etre: 0

  - nom: commande_relivree_a_l_identique
    dimension: unicite
    severite: avertissement
    # Une commande revient dans un autre extrait quand elle a changé. Revenue sans rien changer,
    # c'est un extrait livré deux fois. La ligne entière se compare, lignage exclu.
    sql: |
      WITH lignes AS (
        SELECT _lot, md5(concat_ws('|', *COLUMNS(* EXCLUDE (_fichier, _lot, _charge_le)))) AS empreinte
        FROM raw.erp_vbak)
      SELECT COUNT(*) FROM lignes a
      WHERE {fenetre}
        AND EXISTS (SELECT 1 FROM lignes b WHERE b.empreinte = a.empreinte AND b._lot < a._lot)
    doit_etre: 0

  - nom: concessionnaire_connu
    dimension: integrite
    severite: avertissement
    # La fiche d'un concessionnaire peut arriver un mois après sa première commande : on avertit,
    # et le STAGING d'Analytics Engineering la retrouve au lot suivant.
    sql: |
      SELECT COUNT(*) FROM raw.erp_vbak v
      WHERE {fenetre}
        AND NOT EXISTS (SELECT 1 FROM raw.erp_kna1 k WHERE k.KUNNR = v.KUNNR AND k._lot <= v._lot)
    doit_etre: 0
```

Une commande apparaît une fois par lot : un doublon dans un même extrait est une erreur de l'extraction, et elle bloque. Les numéros SAP ont dix chiffres, les dates huit : la `RAW` garde le texte tel que livré (Cours 03), et c'est ce qui rend la règle possible, les zéros de tête compris. Les deux dernières avertissent. Une commande revient dans un autre extrait quand elle a changé ; revenue à l'identique, lignage exclu, c'est un extrait livré deux fois. Et le concessionnaire d'une commande doit avoir sa fiche dans un lot antérieur ou égal, `k._lot <= v._lot` : une fiche peut arriver un mois après la première commande, et le `STAGING` d'Analytics Engineering la retrouvera.

Piège — hacher une ligne entière. `md5(concat_ws('|', COLUMNS(* EXCLUDE (...))))`, sans l'étoile devant `COLUMNS`, ne rend pas une empreinte par ligne : DuckDB déplie l'expression en autant de colonnes qu'il y en a dans la table, toutes nommées `empreinte`, et la règle ne compare que la première, celle de `MANDT`, qui vaut la même chose partout. Elle trouve alors 27 746 commandes relivrées sur 28 271, toutes sauf celles du premier extrait. L'étoile, `*COLUMNS(...)`, passe les colonnes comme arguments d'un seul appel.

Jugez le contrat sur tout l'historique :

© 2026 iAlexMG. Tous droits réservés.

```
python -m pipeline contrats --livraisons erp
echo %ERRORLEVEL%
```

```
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : erp
INFO    pipeline : étape contrats (52454b33) : début
INFO    pipeline :   respectée  erp_extraits.extrait_du_mois_livre                0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.commandes_du_mois                   380   (attendu >= 300)
INFO    pipeline :   respectée  erp_extraits.commande_unique_dans_son_lot        0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.numeros_au_format_sap               0   (attendu = 0)
WARNING pipeline :   à voir     erp_extraits.commande_relivree_a_l_identique   609   (attendu = 0)
WARNING pipeline :   à voir     erp_extraits.concessionnaire_connu               1   (attendu = 0)
INFO    pipeline : étape contrats (52454b33) : fin, 6 lignes en 1.4 s
INFO    pipeline : pipeline : terminé
0
```

Les deux défauts de l'ERP sortent, en avertissement : 609 commandes relivrées, et une commande dont le concessionnaire n'a pas de fiche à temps. Le code reste 0, et c'est voulu : la `RAW` garde les deux, et le `STAGING` sait les traiter. Jugé sur tout l'historique, pourtant, ce contrat sonnerait à chaque passage pour un extrait livré il y a trois ans. Jugez-le mois par mois, comme le DAG mensuel le fera, sur mars 2023 puis mai 2024 :

```
python -m pipeline contrats --livraisons erp --depuis 2023-03 --jusqua 2023-03
python -m pipeline contrats --livraisons erp --depuis 2024-05 --jusqua 2024-05
```

```
INFO    pipeline : fenêtre : du 2023-03-01 au 2023-03-31
…
INFO    pipeline :   respectée  erp_extraits.commandes_du_mois                   609   (attendu >= 300)
…
WARNING pipeline :   à voir     erp_extraits.commande_relivree_a_l_identique   609   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.concessionnaire_connu               0   (attendu = 0)
…
INFO    pipeline : fenêtre : du 2024-05-01 au 2024-05-31
…
INFO    pipeline :   respectée  erp_extraits.commande_relivree_a_l_identique     0   (attendu = 0)
WARNING pipeline :   à voir     erp_extraits.concessionnaire_connu               1   (attendu = 0)
```

Chaque défaut sonne une fois, le mois où il arrive. La fenêtre de mars 2023 retient ses deux lots, `2023-03` et `2023-03_bis`, parce qu'un lot relivré garde son mois. Demandez enfin le mois d'avril 2026, celui que la `RAW` disait « à jour » :

```
python -m pipeline contrats --livraisons erp --depuis 2026-04 --jusqua 2026-04
echo %ERRORLEVEL%
```

```
INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
INFO    pipeline : livraisons : erp
INFO    pipeline : étape contrats (b69bbdbb) : début
```


© 2026 iAlexMG. Tous droits réservés.

```
ERROR   pipeline :   ROMPUE     erp_extraits.extrait_du_mois_livre                          1   (attendu = 0)
ERROR   pipeline :   ROMPUE     erp_extraits.commandes_du_mois                              0   (attendu >= 300)
INFO    pipeline :   respectée  erp_extraits.commande_unique_dans_son_lot                   0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.numeros_au_format_sap                          0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.commande_relivree_a_l_identique               0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.concessionnaire_connu                         0   (attendu = 0)
ERROR   pipeline : étape contrats (b69bbdbb) : échec, 2 règles bloquantes rompues : erp_extraits.extrait_du_mois_livre,
erp_extraits.commandes_du_mois
ERROR   pipeline : pipeline arrêté : 2 règles bloquantes rompues : erp_extraits.extrait_du_mois_livre,
erp_extraits.commandes_du_mois
1
```

Le mois absent se voit enfin : un mois attendu et non livré, zéro commande, deux règles bloquantes, code 1. Le chargeur ne pouvait pas le dire, parce qu'il ne compare que ce qui est arrivé à ce qu'il a déjà ; le contrat compare ce qui est arrivé à ce qui était promis. Les quatre autres règles ont été jugées quand même : aucune ne s'est arrêtée à la première rompue.

Étape 4 — La télémétrie, de la version 1.0.0 à la 2.0.0

Théorie : `Theorie_11_DE.md`, section 5.

Le deuxième contrat porte sur la télémétrie, et il commence tel qu'on l'aurait écrit en 2024, avant le micrologiciel 2.0. `notepad contrats\telemetrie.yaml` :

```yaml
# telemetrie.yaml : le contrat de la télémétrie des véhicules, telle que la RAW la garde.

contrat: telemetrie
version: 1.0.0
produit: les résumés de sortie des véhicules connectés, dans raw.telemetrie_sorties
producteur: le module télématique des véhicules, par le courtier MQTT et l'abonné de Boréal
consommateurs: Analytics Engineering (les faits de l'après-vente) et Data Science
fenetre:
  colonne: _lot
  forme: lot

regles:
  - nom: archive_du_jour_livree
    dimension: fraicheur
    severite: bloquant
    # Le journal des chargements, et non la table : une archive vide est une archive livrée.
    metrique: periodes_manquantes
    table: pipeline.chargements
    periode: lot
    filtre: table_raw = 'telemetrie_sorties'
```


© 2026 iAlexMG. Tous droits réservés.

```
    pas: jour
    doit_etre: 0
```

La fraîcheur, cette fois, se lit dans le journal des chargements, `pipeline.chargements`, et non dans la table : 41 archives de 2023 sont vides, et une archive vide est une archive livrée. `periode: lot` dit quelle colonne porte le jour, et `filtre` restreint le journal à la télémétrie.

La suite du contrat :

```
  - nom: champs_du_message
    dimension: schema
    severite: bloquant
    metrique: champs_json
    table: raw.telemetrie_sorties
    champ: message
    versions:
      "1.x": [message_id, sernr, micrologiciel, debut, fin, distance_km, regime_moyen, regime_max,
              temp_moteur_max, temp_exterieure, tension_batterie_min, position, codes]
    doit_etre: 0

  - nom: identifiant_sans_conflit
    dimension: unicite
    severite: bloquant
    sql: |
      SELECT COUNT(*) FROM (
        SELECT message->>'message_id' FROM raw.telemetrie_sorties WHERE {fenetre}
        GROUP BY 1 HAVING COUNT(DISTINCT CAST(message AS VARCHAR)) > 1)
    doit_etre: 0

  - nom: numero_de_serie_connu
    dimension: integrite
    severite: avertissement
    sql: |
      SELECT COUNT(*) FROM raw.telemetrie_sorties t
      WHERE {fenetre}
        AND NOT EXISTS (SELECT 1 FROM raw.erp_zserie z WHERE z.SERNR = (t.message->>'sernr'))
    doit_etre: 0
```

La règle de schéma déclare les treize champs du message des micrologiciels 1.x, triés ou non : le module les trie avant de comparer. Deux copies d'un même message sont permises, c'est le prix d'une livraison « au moins une fois » (Cours 09) ; deux contenus différents sous un même identifiant ne le sont pas. Et chaque numéro de série doit exister dans le registre des véhicules de l'ERP, `ZSERIE`. Dans cette dernière requête, les parenthèses autour de `t.message->>'sernr'` ne sont pas décoratives (voir le dépannage).

Jugez le contrat sur le dernier jour d'avril 2025, puis sur le premier jour de mai :

© 2026 iAlexMG. Tous droits réservés.

```
python -m pipeline contrats --livraisons telemetrie --depuis 2025-04-30 --jusqua 2025-04-30
python -m pipeline contrats --livraisons telemetrie --depuis 2025-05-01 --jusqua 2025-05-01
echo %ERRORLEVEL%
```

```
INFO    pipeline : fenêtre : du 2025-04-30 au 2025-04-30
…
INFO    pipeline :   respectée  telemetrie.champs_du_message                              0   (attendu = 0)
…
INFO    pipeline : étape contrats (c6204818) : fin, 4 lignes en 0.4 s
…
INFO    pipeline : fenêtre : du 2025-05-01 au 2025-05-01
INFO    pipeline :   respectée  telemetrie.archive_du_jour_livree                         0   (attendu = 0)
ERROR   pipeline :   ROMPUE     telemetrie.champs_du_message                              4   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.identifiant_sans_conflit                      0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.numero_de_serie_connu                         0   (attendu = 0)
ERROR   pipeline : étape contrats (5b48e889) : échec, 1 règle bloquante rompue : telemetrie.champs_du_message
ERROR   pipeline : pipeline arrêté : 1 règle bloquante rompue : telemetrie.champs_du_message
1
```

Le 30 avril, tout est conforme. Le 1er mai, quatre messages arrivent sous une forme que le contrat ne connaît pas, et le pipeline s'arrête le jour même. C'est ce que personne n'avait vu en 2025. Jugez-le sur tout l'historique, pour mesurer l'étendue :

```
python -m pipeline contrats --livraisons telemetrie
```

```
ERROR   pipeline :   ROMPUE     telemetrie.champs_du_message                          115 807   (attendu = 0)
```

Tous les messages du micrologiciel 2.0 jusqu'au 31 mars 2026. Le contrat rompu ne se contourne pas en baissant la sévérité de la règle : il se renégocie. Le producteur confirme que la forme 2.0 est la nouvelle, les consommateurs disent qu'il leur faut les deux, et le contrat passe en 2.0.0, parce que le changement casse la lecture. Dans `contrats\telemetrie.yaml`, changez la version :

```
version: 2.0.0
```

Et ajoutez la seconde forme du message sous la première, dans `versions`, alignée sur `"1.x"` :

```
      "2.0": [message_id, sernr, micrologiciel, debut, fin, distance_km, heures_moteur,
              regime_moyen, regime_max, temperature_moteur_max, temp_exterieure,
              tension_batterie_min, position, codes]
```

Rejugez le 1er mai, puis tout l'historique :

```
python -m pipeline contrats --livraisons telemetrie --depuis 2025-05-01 --jusqua 2025-05-01
python -m pipeline contrats --livraisons telemetrie
```

```
INFO    pipeline : fenêtre : du 2025-05-01 au 2025-05-01
…
```


© 2026 iAlexMG. Tous droits réservés.

```
INFO    pipeline :   respectée  telemetrie.champs_du_message                                  0   (attendu = 0)
…
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline :   respectée  telemetrie.archive_du_jour_livree                            0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.champs_du_message                                 0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.identifiant_sans_conflit                          0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.numero_de_serie_connu                             0   (attendu = 0)
INFO    pipeline : étape contrats (18748ee0) : fin, 4 lignes en 2.0 s
```

Zéro message hors contrat, sur l'une ou l'autre forme. Les deux passages en 1.0.0 restent inscrits dans la table, avec leur version : le registre garde ce que le contrat disait le jour où il a jugé.

Étape 5 — Lire la plateforme par ses traces

Théorie : `Theorie_11_DE.md`, section 8.

Trois tables racontent maintenant la plateforme : les chargements depuis le Cours 03, les exécutions depuis le Cours 05, les verdicts depuis ce cours. Ouvrez la ligne de commande DuckDB depuis `Projet_Boreal_intermediaire`, et l'entrepôt en lecture seule, comme à l'étape 1. D'abord les passages de l'étape des contrats :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

SELECT etat, lignes, ROUND(duree_s, 1) AS duree_s, message
FROM pipeline.executions WHERE etape = 'contrats' ORDER BY debut;
```

```
┌─────────┬─────────┬──────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  etat   │ lignes  │ duree_s  │                                                 message                                                  │
│ varchar │ int64   │ double   │                                                 varchar                                                  │
├─────────┼─────────┼──────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ réussi  │       0 │      1.3 │ NULL                                                                                                             │
│ réussi  │       6 │      1.4 │ NULL                                                                                                             │
│ réussi  │       6 │      0.2 │ NULL                                                                                                             │
│ réussi  │       6 │      0.2 │ NULL                                                                                                             │
└─────────┴─────────┴──────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```


© 2026 iAlexMG. Tous droits réservés.

```
│ échoué  │   NULL │     0.2 │ ContratRompu : 2 règles bloquantes rompues : erp_extraits.extrait_du_mois_livre,
erp_extraits.commandes_du_mois │
│ réussi  │      4 │     0.4 │ NULL
│
│ échoué  │   NULL │     0.4 │ ContratRompu : 1 règle bloquante rompue : telemetrie.champs_du_message
│
│ échoué  │   NULL │     2.1 │ ContratRompu : 1 règle bloquante rompue : telemetrie.champs_du_message
│
│ réussi  │      4 │     0.4 │ NULL
│
│ réussi  │      4 │     2.0 │ NULL
│
└─────────┴────────┴─────────┴──────────────
```

varchar	int64	int128	varchar
corpus	1	34	NULL
erp	468	166789	2026-03
garanties	255	154553	2026-03
meteo	3	54140	NULL
telemetrie	974	204485	2026-03-31
usine	3	906	NULL

Trois livraisons n'ont pas de lot daté : ce sont des exports complets, qui remplacent le précédent (Cours 03). Reste le lignage. Ramenez les commandes relivrées à leur fichier, par la même empreinte que le contrat. `EXISTS` garde une ligne dès qu'une version plus ancienne porte la même empreinte ; la semi-jointure est au Cours 07 de la formation SQL avancé, section 4 :

```
WITH lignes AS (
    SELECT _fichier, _lot,
           md5(concat_ws('|', *COLUMNS(* EXCLUDE (_fichier, _lot, _charge_le)))) AS empreinte
    FROM raw.erp_vbak)
SELECT a._fichier, COUNT(*) AS relivrees
FROM lignes a
WHERE EXISTS (SELECT 1 FROM lignes b WHERE b.empreinte = a.empreinte AND b._lot < a._lot)
GROUP BY ALL;

SELECT lot, fichier, lignes, LEFT(empreinte, 16) AS empreinte
FROM pipeline.chargements WHERE table_raw = 'erp_vbak' AND lot LIKE '2023-03%' ORDER BY lot;
```

_fichier varchar	relivrees int64
sources/erp/extraits/2023-03_bis/VBAK.csv	609

lot varchar	fichier varchar	lignes int64	empreinte varchar
2023-03	sources/erp/extraits/2023-03/VBAK.csv	609	f8f652a8cb06da34
2023-03_bis	sources/erp/extraits/2023-03_bis/VBAK.csv	609	f8f652a8cb06da34

Toutes les relivrées viennent d'un seul fichier, et le journal des chargements montre pourquoi : deux fichiers de même empreinte, sous deux lots. `_fichier`, écrit dans chaque ligne de la `RAW` depuis le Cours 03, fait remonter une ligne suspecte jusqu'à sa livraison. La règle du contrat ne compare pourtant pas les

© 2026 iAlexMG. Tous droits réservés.

empreintes des fichiers, qui seraient plus simples à lire : les données maîtres de l'ERP, `KNA1`, `MARA`, `MAKT`, reviennent identiques d'un mois à l'autre quand rien n'a changé, et une règle par fichier sonnerait pour elles aussi. C'est une question des exercices.

Quittez DuckDB par `.exit`.

Point d'arrêt. Les contrats de ce que la `RAW` reçoit sont écrits, jugés et lus. Si vous faites ce labo en deux séances, la première s'arrête ici : les étapes 6 à 12 ajoutent l'usine, le rejeu, l'argent, les DAG et l'incident, et ne supposent rien de plus que le dépôt tel qu'il est maintenant.

Étape 6 — Le contrat de l'historien

Théorie : `Theorie_11_DE.md`, section 3.

L'historien de l'usine ne passe pas par la `RAW` : ses points vivent au bronze, en Parquet, depuis le Cours 02, et c'est là que son contrat les juge. `notepad contrats\usine_historien.yaml`. L'en-tête, et les vues que les règles partagent :

```
# usine_historien.yaml : le contrat de l'export quotidien de l'historien, au bronze.

contrat: usine_historien
version: 1.0.0
produit: l'export quotidien de l'historien du SCADA, dans plateforme/lac/bronze/usine/historien/
producteur: l'intégrateur du SCADA de l'usine de Val-des-Aulnes
consommateurs: Data Engineering, pour les séries de l'argent (Cours 08) ; la maintenance de l'usine
fenetre:
  colonne: jour
  forme: date

# Les vues que les règles partagent : les points du jour, valeurs de départ exclues (Cours 08), le
# référentiel des étiquettes, et les journées où une ligne a produit au moins un véhicule.
preparation: |
  CREATE OR REPLACE TEMP VIEW points AS
  SELECT * FROM read_parquet('plateforme/lac/bronze/usine/historien/**/*.parquet',
                             hive_partitioning = true)
  WHERE CAST(Timestamp AS DATE) = jour;
  CREATE OR REPLACE TEMP TABLE etiquettes AS
  SELECT * FROM read_csv('sources/usine/referentiel/etiquettes.csv');
  CREATE OR REPLACE TEMP TABLE jours_de_production AS
  SELECT jour FROM points WHERE TagName LIKE '%.CNV.COMPTEUR' AND Quality = 192
  GROUP BY jour HAVING MAX(Value) > 0;
```

Le bloc `preparation` pose trois objets avant les règles. `points` lit le bronze et écarte les valeurs de départ, qui portent l'horodatage de la veille (Cours 08) : sans ce filtre, chaque journée compterait les valeurs de départ du lendemain. `etiquettes` lit le référentiel, avec la plage et la bande morte de chaque capteur.

© 2026 iAlexMG. Tous droits réservés.

`jours_de_production` retient les journées où le compteur d'une ligne a bougé. Les chemins partent de `Projet_Boreal_intermediaire`, parce que le pipeline y travaille, d'où qu'on le lance.

Les quatre premières règles :

```yaml
regles:
  - nom: export_du_jour_livre
    dimension: fraicheur
    severite: bloquant
    metrique: periodes_manquantes
    table: points
    pas: jour
    doit_etre: 0

  - nom: etiquette_au_referentiel
    dimension: integrite
    severite: bloquant
    metrique: invalides
    table: points
    valide: TagName IN (SELECT etiquette FROM etiquettes)
    doit_etre: 0

  - nom: bonne_qualite_dans_la_plage
    dimension: validite
    severite: bloquant
    sql: |
      SELECT COUNT(*) FROM points p JOIN etiquettes e ON e.etiquette = p.TagName
      WHERE {fenetre} AND p.Quality = 192 AND NOT p.Value BETWEEN e.plage_min AND e.plage_max
    doit_etre: 0

  - nom: points_douteux
    dimension: validite
    severite: avertissement
    metrique: lignes
    table: points
    filtre: Quality = 64
    doit_etre: 0
```

La fraîcheur, par jour cette fois, et sur la colonne `jour` du bronze. L'intégrité : chaque étiquette au référentiel. La validité : une valeur de bonne qualité, code 192, dans la plage de son capteur ; les valeurs douteuses, code 64, ont le droit d'en sortir, c'est pour cela qu'elles sont douteuses. Puis les points douteux eux-mêmes, comptés : une règle de validité sur la métrique `lignes`, avec un filtre.

© 2026 iAlexMG. Tous droits réservés.

Les deux dernières, pour cette étape :

```yaml
  - nom: coupures_de_communication
    dimension: completude
    severite: avertissement
    metrique: lignes
    table: points
    filtre: Quality = 0
    doit_etre: 0

  - nom: capteur_fige
    dimension: validite
    severite: avertissement
    # Une mesure (bande morte non nulle) qui garde la même valeur non nulle toute une journée de
    # production. Le zéro est écarté : c'est la valeur d'un équipement à l'arrêt.
    sql: |
      SELECT COUNT(*) FROM (
        SELECT p.TagName, p.jour FROM points p JOIN etiquettes e ON e.etiquette = p.TagName
        WHERE {fenetre} AND e.bande_morte > 0 AND p.Quality = 192
          AND p.jour IN (SELECT jour FROM jours_de_production)
        GROUP BY ALL HAVING COUNT(DISTINCT p.Value) = 1 AND MIN(p.Value) <> 0)
    doit_etre: 0
```

Les coupures de communication laissent des points de code 0, sans valeur : une règle de complétude. Le capteur figé est une mesure qui ne bouge plus. Une étiquette dont la bande morte est nulle est un état ou un compteur, qui a le droit de rester immobile ; une mesure, non. Et le zéro est écarté, parce que c'est la valeur d'un équipement à l'arrêt : sans cette condition, la règle trouve des centaines de journées où un équipement arrêté mesure, fidèlement, zéro.

Les points douteux et les coupures sont en `avertissement` pour l'instant. C'est un choix à éprouver, et l'étape 8 le fera. Jugez le contrat sur tout l'historique, depuis `01_de` :

```
python -m pipeline contrats --livraisons usine
```

```
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : usine
INFO    pipeline : étape contrats (ae045b7b) : début
INFO    pipeline :   respectée  usine_historien.export_du_jour_livre                    0   (attendu = 0)
INFO    pipeline :   respectée  usine_historien.etiquette_au_referentiel                0   (attendu = 0)
INFO    pipeline :   respectée  usine_historien.bonne_qualite_dans_la_plage             0   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.points_douteux                         64   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.coupures_de_communication             529   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.capteur_fige                            1   (attendu = 0)
INFO    pipeline : étape contrats (ae045b7b) : fin, 6 lignes en 1.9 s
INFO    pipeline : pipeline : terminé
```


© 2026 iAlexMG. Tous droits réservés.

Aucun export ne manque, aucune étiquette n'est inconnue, aucune bonne valeur ne sort de sa plage. Les 64 points douteux et les 529 points perdus aux coupures sont ceux que le Cours 08 a comptés. Le capteur figé est `PEI.CAB.HUMIDITE`, l'humidité de la cabine de peinture, le 8 octobre 2025 : un exercice vous le fera retrouver.

Étape 7 — La pression, bâtie sur la vraie sortie

Théorie : `Theorie_11_DE.md`, section 6.

Reste la règle qui n'a pas d'attendu fixe : la pression au collecteur du réseau d'air, `UTL.AIR.PRESSION`, qui devient instable quand le compresseur 2 dérive. Une telle règle ne s'écrit pas d'intuition. On la construit dans la ligne de commande DuckDB, on la lance sur des mois de données, et on regarde chaque alerte.

Ouvrez la ligne de commande DuckDB depuis `Projet_Boreal_intermediaire`. Cette fois, aucun `ATTACH` : tout se lit au bronze, et la ligne de commande peut rester ouverte pendant que le pipeline tourne, puisqu'elle ne touche pas à l'entrepôt. Posez les mêmes objets que le contrat :

```
CREATE OR REPLACE VIEW points AS
SELECT * FROM read_parquet('plateforme/lac/bronze/usine/historien/**/*.parquet',
                           hive_partitioning = true)
WHERE CAST(Timestamp AS DATE) = jour;
CREATE OR REPLACE TABLE jours_de_production AS
SELECT jour FROM points WHERE TagName LIKE '%.CNV.COMPTEUR' AND Quality = 192
GROUP BY jour HAVING MAX(Value) > 0;

-- Attendu : 121 journées de production, du 2025-10-01 au 2026-03-31.
SELECT COUNT(*) AS journees, MIN(jour) AS premiere, MAX(jour) AS derniere
FROM jours_de_production;
```

journees int64	premiere date	derniere date
121	2025-10-01	2026-03-31

Première idée, la plus naturelle : l'écart-type de la pression de 7 h à 14 h, les jours de production, là où l'usine tourne à coup sûr.

```
CREATE OR REPLACE TABLE journees AS
SELECT jour, STDDEV_SAMP(Value) AS ecart
FROM points
WHERE TagName = 'UTL.AIR.PRESSION' AND Quality = 192
  AND HOUR(Timestamp) BETWEEN 7 AND 14 AND jour IN (FROM jours_de_production)
GROUP BY jour;
```


© 2026 iAlexMG. Tous droits réservés.

Puis le jugement : chaque journée contre la médiane des vingt journées de production qui la précèdent, avec au moins cinq journées de référence, et une alerte au-delà du double. La médiane glissante est une fenêtre au cadre explicite, qui s'arrête à la veille pour ne pas compter la journée jugée : les cadres sont au Cours 04 de la formation SQL avancé, section 6, et la médiane à son Cours 09, section 8.

```
SELECT jour, ROUND(ecart, 1) AS ecart, ROUND(reference, 1) AS reference
FROM (SELECT jour, ecart, MEDIAN(ecart) OVER vingt AS reference, COUNT(*) OVER vingt AS jours
      FROM journees
      WINDOW vingt AS (ORDER BY jour ROWS BETWEEN 20 PRECEDING AND 1 PRECEDING))
WHERE jours >= 5 AND ecart > 2 * reference
ORDER BY jour;
```

jour date	ecart double	reference double
2025-10-20	15.1	4.0
2025-10-21	22.4	4.0
2025-10-22	28.8	4.2
2025-10-23	37.2	4.3
2025-10-24	44.6	4.4
2025-10-27	58.1	4.5
2025-10-28	60.7	4.8
2025-12-22	209.3	3.6
2025-12-23	177.2	3.6
2026-01-07	222.3	3.6
2026-01-08	208.4	3.7
2026-01-09	219.6	4.2
2026-01-13	208.5	5.0
2026-01-14	219.7	5.0
2026-01-16	228.6	5.2

Quinze alertes. Les sept premières tombent pendant la dérive, du 20 au 28 octobre : ce sont les vraies. Les huit suivantes tombent en décembre et en janvier, sur une usine qui va très bien, et avec des écarts-types plus grands encore. Avant de corriger quoi que ce soit, regardez-en une, heure par heure :

```
SELECT HOUR(Timestamp) AS heure, COUNT(*) AS points, ROUND(MIN(Value)) AS mini,
       ROUND(MAX(Value)) AS maxi
FROM points
WHERE TagName = 'UTL.AIR.PRESSION' AND Quality = 192 AND jour = DATE '2026-01-07'
  AND HOUR(Timestamp) BETWEEN 6 AND 15
GROUP BY ALL ORDER BY ALL;
```


© 2026 iAlexMG. Tous droits réservés.

heure int64	points int64	mini double	maxi double
6	9	344.0	706.0
7	1	702.0	702.0
8	2	699.0	708.0
9	1	706.0	706.0
10	1	707.0	707.0
11	5	596.0	713.0
12	30	155.0	571.0
13	18	42.0	149.0
14	5	13.0	36.0
15	2	3.0	8.0

La pression tient autour de 700 kPa jusqu'à 11 h, puis s'effondre : à 14 h, elle n'est plus qu'à quelques dizaines de kPa. Le 7 janvier est une journée courte ; les lignes s'arrêtent vers 11 h, les compresseurs avec elles, et le réseau se vide. Un écart-type de 222 kPa, pour une usine qui s'est simplement arrêtée plus tôt. Les huit fausses alertes sont toutes de cette sorte. La fenêtre de 7 h à 14 h supposait que l'usine tourne à ces heures-là ; il faut plutôt demander à l'usine si elle tourne.

Les lignes le disent : chacune publie son état de marche, `LMN.CNV.MARCHE`, `LMM.CNV.MARCHE` et `LHR.CNV.MARCHE`, 1 quand elle tourne. Attachez à chaque point de pression l'état des trois lignes au même instant, par une jointure `ASOF`, qui prend pour chaque point le dernier état publié avant lui, et ne gardez que les points pris pendant qu'au moins une ligne tourne :

```
CREATE OR REPLACE TABLE journees AS
WITH marche AS (
  SELECT TagName, Timestamp, Value AS etat FROM points
  WHERE TagName IN ('LMN.CNV.MARCHE', 'LMM.CNV.MARCHE', 'LHR.CNV.MARCHE') AND Quality = 192
), pression AS (
  SELECT p.jour, p.Value,
         COALESCE(a.etat, 0) + COALESCE(b.etat, 0) + COALESCE(c.etat, 0) AS lignes
  FROM (SELECT * FROM points WHERE TagName = 'UTL.AIR.PRESSION' AND Quality = 192) p
  ASOF LEFT JOIN (FROM marche WHERE TagName = 'LMN.CNV.MARCHE') a
         ON p.Timestamp >= a.Timestamp
  ASOF LEFT JOIN (FROM marche WHERE TagName = 'LMM.CNV.MARCHE') b
         ON p.Timestamp >= b.Timestamp
  ASOF LEFT JOIN (FROM marche WHERE TagName = 'LHR.CNV.MARCHE') c
         ON p.Timestamp >= c.Timestamp
)
SELECT jour, STDDEV_SAMP(Value) AS ecart, COUNT(*) AS points FROM pression
```


© 2026 iAlexMG. Tous droits réservés.

```
WHERE lignes > 0 AND jour IN (SELECT jour FROM jours_de_production)
GROUP BY jour;
```

Relancez la requête du jugement, sans y toucher :

jour date	ecart double	reference double
2025-10-20	15.1	4.1
2025-10-21	22.2	4.2
2025-10-22	28.1	4.4
2025-10-23	36.8	4.6
2025-10-24	45.7	4.7
2025-10-27	58.4	4.8
2025-10-28	60.4	4.8

Sept alertes, toutes pendant la dérive, zéro fausse. Regardez maintenant ce que la règle ne voit pas, autour de la dérive, qui commence le 15 octobre :

```
SELECT jour, ROUND(ecart, 1) AS ecart, points FROM journees
WHERE jour BETWEEN DATE '2025-10-14' AND DATE '2025-10-21' ORDER BY jour;
```

jour date	ecart double	points int64
2025-10-14	4.8	15
2025-10-15	4.2	13
2025-10-16	3.4	15
2025-10-17	4.6	28
2025-10-20	15.1	118
2025-10-21	22.2	161

Les 15, 16 et 17 octobre ne se distinguent pas par leur écart-type : trois jours manqués. Le 17, pourtant, la pression a écrit 28 points pendant la marche, plus du double des 12 d'une journée ordinaire : la bande morte de l'historien écrit un point chaque fois que la valeur bouge assez (Cours 08), et la pression commence à bouger. Une règle sur le nombre de points aurait alerté un jour ouvrable plus tôt. C'est une question des exercices.

La règle est prête ; ajoutez-la au contrat, à la fin de `contrats\usine_historien.yaml` :

```
  - nom: pression_instable
    dimension: distribution
    severite: avertissement
```


© 2026 iAlexMG. Tous droits réservés.

```
    sql: |
      WITH marche AS (
        SELECT TagName, Timestamp, Value AS etat FROM points
        WHERE TagName IN ('LMN.CNV.MARCHE', 'LMM.CNV.MARCHE', 'LHR.CNV.MARCHE') AND Quality = 192
      ), pression AS (
        SELECT p.jour, p.Value,
               COALESCE(a.etat, 0) + COALESCE(b.etat, 0) + COALESCE(c.etat, 0) AS lignes
        FROM (SELECT * FROM points WHERE TagName = 'UTL.AIR.PRESSION' AND Quality = 192) p
        ASOF LEFT JOIN (FROM marche WHERE TagName = 'LMN.CNV.MARCHE') a
               ON p.Timestamp >= a.Timestamp
        ASOF LEFT JOIN (FROM marche WHERE TagName = 'LMM.CNV.MARCHE') b
               ON p.Timestamp >= b.Timestamp
        ASOF LEFT JOIN (FROM marche WHERE TagName = 'LHR.CNV.MARCHE') c
               ON p.Timestamp >= c.Timestamp
      ), journees AS (
        SELECT jour, STDDEV_SAMP(Value) AS ecart FROM pression
        WHERE lignes > 0 AND jour IN (SELECT jour FROM jours_de_production)
        GROUP BY jour
      ), jugees AS (
        SELECT jour, ecart, MEDIAN(ecart) OVER vingt AS reference, COUNT(*) OVER vingt AS jours
        FROM journees
        WINDOW vingt AS (ORDER BY jour ROWS BETWEEN 20 PRECEDING AND 1 PRECEDING)
      )
      SELECT COUNT(*) FROM jugees
      WHERE {fenetre} AND jours >= 5 AND ecart > 2 * reference
    doit_etre: 0
```

C’est la même requête, en une seule, avec `{fenetre}` posé sur le jugement et non sur la construction : la médiane des vingt journées précédentes se calcule sur tout le bronze, même quand la fenêtre ne juge qu’une journée. Jugez le contrat, depuis `01_de` :

```
python -m pipeline contrats --livraisons usine

…
WARNING pipeline :   à voir     usine_historien.capteur_fige                          1   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.pression_instable                     7   (attendu = 0)
INFO    pipeline : étape contrats (1e023935) : fin, 7 lignes en 2.1 s
```

Les sept journées de la dérive. La première, le 20 octobre 2025, tombe huit jours avant le déclenchement du compresseur 2, deux jours avant l’alerte de vibration du Cours 09.

Étape 8 — Rejouer 182 jours, et compter les alertes

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_11_DE.md`, sections 4 et 7.

Jugé sur tout l'historique, un contrat dit ce qui est vrai en bloc. Il ne dit pas combien de fois il aurait sonné. Pour le savoir, il faut le rejouer comme le DAG quotidien l'aurait fait, une journée à la fois. `notepad rejouer_contrats.py`, depuis `01_de` :

```
# -*- coding: utf-8 -*-
"""rejouer_contrats.py : les contrats d'une livraison rejoués tranche par tranche.

Usage, depuis 01_de :
    python rejouer_contrats.py usine --depuis 2025-10-01 --jusqua 2026-03-31
    python rejouer_contrats.py erp --depuis 2022-01 --jusqua 2026-03 --par-mois
"""
import argparse
import logging
import os
import sys
from datetime import date, datetime, timedelta, timezone

from pipeline import contrats
from pipeline.config import projet
from pipeline.entrepot import ouvrir
from pipeline.fenetre import Fenetre
from pipeline.journal import LOG


def tranches(depuis: str, jusqua: str, par_mois: bool):
    """Les fenêtres à juger, une par jour ou une par mois, bornes comprises."""
    fin = Fenetre.depuis_la_ligne_de_commande(jusqua, jusqua).jusqua
    jour = Fenetre.depuis_la_ligne_de_commande(depuis, depuis).depuis
    while jour <= fin:
        if par_mois:
            texte = jour.strftime("%Y-%m")
            yield Fenetre.depuis_la_ligne_de_commande(texte, texte)
            jour = date(jour.year + jour.month // 12, jour.month % 12 + 1, 1)
        else:
            yield Fenetre(jour, jour)
            jour += timedelta(days=1)
```

`tranches()` découpe la période en fenêtres d'un jour, ou d'un mois avec `--par-mois`, par la même classe `Fenetre` que le pipeline. Puis le rejeu, et son bilan :

```
def main() -> int:
    analyseur = argparse.ArgumentParser(description="Rejoue les contrats d'une livraison.")
    analyseur.add_argument("livraison", help="erp, telemetrie, usine ou argent")
```


© 2026 iAlexMG. Tous droits réservés.

```
    analyseur.add_argument("--depuis", required=True)
    analyseur.add_argument("--jusqua", required=True)
    analyseur.add_argument("--par-mois", action="store_true", help="une tranche par mois")
    options = analyseur.parse_args()

    os.chdir(projet())
    LOG.setLevel(logging.CRITICAL)        # le rejeu parle par son bilan, pas par ses journaux
    depart = datetime.now(timezone.utc)
    con = ouvrir()
    try:
        for fenetre in tranches(options.depuis, options.jusqua, options.par_mois):
            try:
                contrats.executer(con, fenetre, [options.livraison])
            except contrats.ContratRompu:
                pass                      # une tranche rompue est un résultat, pas une panne
        bilan = con.execute("""
            SELECT regle, severite, COUNT(*) AS tranches,
                   COUNT(*) FILTER (WHERE NOT respectee) AS hors_cible,
                   string_agg(SUBSTR(fenetre, 4, 10), ' ' ORDER BY fenetre)
                       FILTER (WHERE NOT respectee) AS lesquelles
            FROM pipeline.controles WHERE controle_le >= ?
            GROUP BY ALL ORDER BY hors_cible DESC, regle""", [depart]).fetchall()
    finally:
        con.close()
    duree = (datetime.now(timezone.utc) - depart).total_seconds()
    print(f"{bilan[0][2] if bilan else 0} tranches jugées, en {duree:.0f} s.")
    for regle, severite, _, hors_cible, lesquelles in bilan:
        print(f"  {regle:40} {severite:13} {hors_cible:4} hors cible"
              + (f" : {lesquelles}" if hors_cible and hors_cible <= 12 else ""))
    # Une règle de suivi s'inscrit sans prévenir personne : seules les autres font une alerte.
    alertes = sum(h for _, s, _, h, _ in bilan if s != "suivi")
    bloquantes = sum(h for _, s, _, h, _ in bilan if s == "bloquant")
    print(f"Alertes envoyées : {alertes}, dont {bloquantes} bloquantes.")
    return 0


if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    sys.exit(main())
```


© 2026 iAlexMG. Tous droits réservés.

Chaque tranche passe par `contrats.executer()`, exactement comme une exécution du pipeline, et ses verdicts s'inscrivent dans `pipeline.controles`. Une tranche rompue n'arrête pas le rejeu : c'est un résultat. Le bilan se lit dans la table, pour les seules lignes écrites depuis le début du rejeu, et compte les alertes : toute règle rompue qui n'est pas en suivi.

Rejouez l'usine sur ses 182 jours, du 1er octobre 2025 au 31 mars 2026. Comptez deux minutes :

```
python rejouer_contrats.py usine --depuis 2025-10-01 --jusqua 2026-03-31
```

```
182 tranches jugées, en 114 s.
  points_douteux                 avertissement   55 hors cible
  coupures_de_communication      avertissement   44 hors cible
  pression_instable              avertissement    7 hors cible : 2025-10-20 2025-10-21 2025-10-22 2025-10-23 2025-10-24
2025-10-27 2025-10-28
  capteur_fige                   avertissement    1 hors cible : 2025-10-08
  bonne_qualite_dans_la_plage    bloquant         0 hors cible
  etiquette_au_referentiel       bloquant         0 hors cible
  export_du_jour_livre           bloquant         0 hors cible
Alertes envoyées : 107, dont 0 bloquantes.
```

Cent sept alertes en six mois. Classez-les, comme la théorie le demande :

Classe	Les alertes du rejeu
Vraies	la pression, 7 journées, toutes pendant la dérive ; le capteur figé, 1 journée
Fausses	aucune
Manquées	les 15, 16 et 17 octobre, début de la dérive, que la pression ne voit pas encore
Vraies mais inutiles	les points douteux, 55 journées ; les coupures, 44 journées

Les points douteux et les coupures sont réels : l'historien les marque lui-même, et les séries de l'argent les comptent déjà (Cours 08). Mais une alerte qui sonne un jour sur trois pour un défaut connu, et après laquelle personne n'a rien à faire, finit par ne plus être lue, et emporte avec elle les huit qui comptaient. Passez ces deux règles en suivi : dans `contrats\usine_historien.yaml`, remplacez `severite: avertissement` par `severite: suivi`, dans `points_douteux` et dans `coupures_de_communication`, et nulle part ailleurs. Rejouez :

```
python rejouer_contrats.py usine --depuis 2025-10-01 --jusqua 2026-03-31
```

```
182 tranches jugées, en 115 s.
  points_douteux                 suivi           55 hors cible
  coupures_de_communication      suivi           44 hors cible
  pression_instable              avertissement    7 hors cible : 2025-10-20 2025-10-21 2025-10-22 2025-10-23 2025-10-24
2025-10-27 2025-10-28
  capteur_fige                   avertissement    1 hors cible : 2025-10-08
```


© 2026 iAlexMG. Tous droits réservés.

```
  bonne_qualite_dans_la_plage             bloquant         0 hors cible
  etiquette_au_referentiel                bloquant         0 hors cible
  export_du_jour_livre                    bloquant         0 hors cible
Alertes envoyées : 8, dont 0 bloquantes.
```

Huit alertes, toutes vraies. Les deux règles de suivi n'ont pas disparu : elles s'inscrivent toujours dans `pipeline.controles`, jour après jour, et un tableau de bord hebdomadaire les lira. Elles ont seulement cessé de réveiller quelqu'un.

Le même rejeu vaut pour l'ERP, mois par mois, comme le DAG mensuel l'aurait fait depuis 2022 :

```
python rejouer_contrats.py erp --depuis 2022-01 --jusqua 2026-03 --par-mois
```

```
51 tranches jugées, en 8 s.
  commande_relivree_a_l_identique         avertissement    1 hors cible : 2023-03-01
  concessionnaire_connu                   avertissement    1 hors cible : 2024-05-01
  commande_unique_dans_son_lot            bloquant         0 hors cible
  commandes_du_mois                       bloquant         0 hors cible
  extrait_du_mois_livre                   bloquant         0 hors cible
  numeros_au_format_sap                   bloquant         0 hors cible
Alertes envoyées : 2, dont 0 bloquantes.
```

Deux alertes en 51 mois, chacune le mois de son défaut.

Étape 9 — Le contrat de ce que Data Engineering livre

Théorie : `Theorie_11_DE.md`, section 2.

Les trois premiers contrats jugent ce que Boréal reçoit. Le dernier juge ce que Data Engineering livre aux formations suivantes : les séries horaires de l'usine du Cours 08, dans le schéma `argent`. Cette fois, le producteur, c'est vous. `notepad contrats\argent_usine.yaml` :

```yaml
# argent_usine.yaml : le contrat des séries de l'usine que Data Engineering livre.

contrat: argent_usine
version: 1.0.0
produit: les séries horaires de l'usine, dans le schéma argent de l'entrepôt
producteur: Data Engineering (01_de/sql/argent_usine.sql)
consommateurs: Analytics Engineering (l'étoile de l'usine), Data Science, Business Intelligence
fenetre:
  colonne: heure
  forme: date

regles:
  - nom: une_ligne_par_etiquette_et_par_heure
```


© 2026 iAlexMG. Tous droits réservés.

```
    dimension: unicite
    severite: bloquant
    metrique: doublons
    table: argent.usine_horaire
    colonnes: [etiquette, heure]
    doit_etre: 0

  - nom: couverture_bornee
    dimension: validite
    severite: bloquant
    metrique: invalides
    table: argent.usine_horaire
    valide: couverture_s BETWEEN 0 AND 3600
    doit_etre: 0
```

La fenêtre est une date, la colonne `heure`. Une ligne par étiquette et par heure, une couverture entre 0 et 3 600 secondes : ce sont les promesses de la section 3 de `LIVRAISON.md`, écrites pour qu'une machine les vérifie.

La suite :

```
  - nom: valeur_des_qu_une_seconde_est_couverte
    dimension: completude
    severite: bloquant
    metrique: invalides
    table: argent.usine_horaire
    valide: (valeur IS NULL) = (couverture_s = 0)
    doit_etre: 0

  - nom: production_egale_au_cumul
    dimension: coherence
    severite: bloquant
    sql: |
      SELECT COUNT(*) FROM (
        SELECT etiquette, CAST(heure AT TIME ZONE 'America/Toronto' AS DATE)
        FROM argent.production_horaire WHERE {fenetre}
        GROUP BY 1, 2 HAVING SUM(production) <> MAX(cumul))
    doit_etre: 0

  - nom: moyenne_dans_la_plage_du_capteur
    dimension: validite
    severite: avertissement
    sql: |
      SELECT COUNT(*) FROM argent.usine_horaire u
```


© 2026 iAlexMG. Tous droits réservés.

```
      JOIN raw.usine_etiquettes e ON e.etiquette = u.etiquette
      WHERE {fenetre}
        AND NOT u.valeur BETWEEN CAST(e.plage_min AS DOUBLE) AND CAST(e.plage_max AS DOUBLE)
    doit_etre: 0
```

Une valeur existe dès qu'une seconde est couverte, et seulement alors : `(valeur IS NULL) = (couverture_s = 0)` compare deux vérités. La production d'une journée égale le dernier cumul du compteur, une règle de cohérence entre deux calculs qui doivent dire la même chose. Et une moyenne horaire doit rester dans la plage de son capteur, en avertissement seulement. Jugez-le :

```
python -m pipeline contrats --livraisons argent
```

```
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : argent
INFO    pipeline : étape contrats (0a966c94) : début
INFO    pipeline :   respectée  argent_usine.une_ligne_par_etiquette_et_par_heure            0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.couverture_bornee                               0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.valeur_des_qu_une_seconde_est_couverte          0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.production_egale_au_cumul                       0   (attendu = 0)
WARNING pipeline :   à voir     argent_usine.moyenne_dans_la_plage_du_capteur               10   (attendu = 0)
INFO    pipeline : étape contrats (0a966c94) : fin, 5 lignes en 1.4 s
```

Quatre promesses tenues. La cinquième règle trouve dix heures dont la moyenne sort de la plage du capteur : le Cours 08 a choisi de compter une valeur douteuse plutôt que de l'écarter, et une moyenne peut alors sortir de la plage. Le contrat ne corrige pas ce choix : il le rend visible à ceux qui lisent l'argent.

Jugez maintenant les quatre contrats, sur tout l'historique :

```
python -m pipeline contrats
```

```
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : toutes
INFO    pipeline : étape contrats (dfa9deb4) : début
INFO    pipeline :   respectée  argent_usine.une_ligne_par_etiquette_et_par_heure            0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.couverture_bornee                               0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.valeur_des_qu_une_seconde_est_couverte          0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.production_egale_au_cumul                       0   (attendu = 0)
WARNING pipeline :   à voir     argent_usine.moyenne_dans_la_plage_du_capteur               10   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.extrait_du_mois_livre                           0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.commandes_du_mois                              380   (attendu >= 300)
INFO    pipeline :   respectée  erp_extraits.commande_unique_dans_son_lot                    0   (attendu = 0)
INFO    pipeline :   respectée  erp_extraits.numeros_au_format_sap                           0   (attendu = 0)
WARNING pipeline :   à voir     erp_extraits.commande_relivree_a_l_identique              609   (attendu = 0)
WARNING pipeline :   à voir     erp_extraits.concessionnaire_connu                          1   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.archive_du_jour_livree                           0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.champs_du_message                                0   (attendu = 0)
```


© 2026 iAlexMG. Tous droits réservés.

```
INFO    pipeline :   respectée  telemetrie.identifiant_sans_conflit                    0   (attendu = 0)
INFO    pipeline :   respectée  telemetrie.numero_de_serie_connu                      0   (attendu = 0)
INFO    pipeline :   respectée  usine_historien.export_du_jour_livre                  0   (attendu = 0)
INFO    pipeline :   respectée  usine_historien.etiquette_au_referentiel              0   (attendu = 0)
INFO    pipeline :   respectée  usine_historien.bonne_qualite_dans_la_plage           0   (attendu = 0)
INFO    pipeline :   hors cible usine_historien.points_douteux                       64   (attendu = 0)
INFO    pipeline :   hors cible usine_historien.coupures_de_communication           529   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.capteur_fige                         1   (attendu = 0)
WARNING pipeline :   à voir     usine_historien.pression_instable                    7   (attendu = 0)
INFO    pipeline : étape contrats (dfa9deb4) : fin, 22 lignes en 3.1 s
INFO    pipeline : pipeline : terminé
```

Vingt-deux règles, en quelques secondes. Le code est 0 : sur tout l'historique, rien ne bloque, et tout ce qui sonne est connu, compté et expliqué.

Étape 10 — Les contrats dans les DAG

Théorie : `Theorie_11_DE.md`, section 4.

Un contrat qu'on lance à la main ne protège que le jour où l'on y pense. Sa place est dans le DAG qui charge la livraison, entre le chargement et ce qui se construit dessus. Ouvrez `dags\boreal_quotidien.py`, celui du Cours 06, depuis `01_de`. Ajoutez la tâche des contrats sous celle du corpus, avant la place de dbt :

```
    # Les contrats de la journée : l'historien et la télémétrie qu'elle a livrés (Cours 11). Une
    # règle bloquante rompue fait échouer la tâche, et dbt_build n'a pas lieu : la RAW garde ce qui
    # est arrivé, mais rien ne se construit dessus tant que le défaut n'est pas compris. Une règle
    # d'avertissement s'écrit au journal de la tâche et dans pipeline.controles, sans rien arrêter.
    contrats = BashOperator(
        task_id="contrats",
        bash_command=f"{PIPELINE} contrats --livraisons usine,telemetrie "
                     f"--depuis {{{{ ds }}}} --jusqua {{{{ ds }}}}",
        pool=ENTREPOT,
    )
```

Remplacez la dernière ligne de la fonction, qui enchaîne les tâches :

```
    bronze >> raw >> corpus >> contrats >> dbt_build
```

Et dans la docstring, le premier paragraphe compte une tâche de plus :

```
Cinq tâches, dans l'ordre : le bronze de l'usine, la couche RAW limitée à la journée de
l'exécution, le dépôt du corpus, les contrats de la journée, puis la place que dbt prendra. Chacune
appelle la porte d'entrée du pipeline. Airflow ordonnance, le pipeline travaille.
```


© 2026 iAlexMG. Tous droits réservés.

La tâche juge les deux livraisons quotidiennes, l'usine et la télémétrie, sur la journée de l'exécution : les quatre accolades de la chaîne `f` en laissent deux, `{{ ds }}`, comme pour la tâche `raw` du Cours 06. Elle prend le pool de l'entrepôt, puisqu'elle y écrit ses verdicts.

Puis `dags\boreal_mensuel.py`. Nommez la tâche existante, pour pouvoir la relier, en changeant sa première ligne :

```
raw_du_mois = BashOperator(
```

Ajoutez sous elle, dans la fonction, la tâche des contrats du mois et l'ordre des deux :

```
    # Le contrat des extraits de l'ERP, jugé sur le mois seulement (Cours 11) : un défaut connu d'un
    # mois ancien ne sonne qu'une fois, le jour où son mois arrive. Un extrait absent ou tronqué fait
    # échouer la tâche ; une commande relivrée ou un concessionnaire en retard s'écrit et passe.
    contrats_du_mois = BashOperator(
        task_id="contrats_du_mois",
        bash_command=f"{PIPELINE} contrats --livraisons erp --depuis {MOIS} --jusqua {MOIS}",
        pool="entrepot",
    )

    raw_du_mois >> contrats_du_mois
```

Dans sa docstring, le premier paragraphe devient :

```
Deux tâches. La première charge dans la RAW les extraits du mois de son intervalle de données :
les neuf fichiers de l'ERP et les cinq des garanties. La seconde juge le contrat des extraits de
l'ERP sur ce même mois.
```

Remplacez aussi « Pourquoi une seule tâche, alors que le quotidien en a quatre. » par « Pourquoi deux tâches seulement, alors que le quotidien en a cinq. », et la fin du commentaire de `end_date`, qui annonçait ce cours :

```
    # La borne de fin est inclusive : mise au 2026-04-01, elle ferait naître une quatrième
    # exécution, celle d'avril. Le chargeur ne trouverait aucun fichier dans sa fenêtre et rendrait
    # zéro ligne, sans planter. Depuis le Cours 11, la tâche des contrats le voit et échoue.
```

La pile Airflow du Cours 06 doit tourner. Dans Ubuntu, depuis `Projet_Boreal_intermediaire/plateforme`, démarrez ses services s'ils sont arrêtés, puis vérifiez que les deux fichiers se lisent :

```
docker compose up -d airflow-apiserver airflow-scheduler airflow-dag-processor airflow-triggerer
docker compose exec airflow-scheduler airflow dags list-import-errors
```

Attendu : `No data found`, une demi-minute après l'enregistrement des fichiers. L'image n'a rien à installer : Airflow dépend lui-même de PyYAML.

Les exécutions du Cours 06 sont finies, et Airflow n'ajoute pas une tâche neuve à une exécution finie. Pour voir les contrats tourner sur les mêmes tranches, effacez l'historique des deux DAG, qui reviennent d'eux-mêmes une demi-minute plus tard. L'effacement n'emporte que l'historique des exécutions, pas les données de l'entrepôt :

© 2026 iAlexMG. Tous droits réservés.

```
docker compose exec airflow-scheduler airflow dags delete boreal_mensuel -y
docker compose exec airflow-scheduler airflow dags delete boreal_quotidien -y
```

Attendez que les deux reviennent dans `airflow dags list`, puis relancez la journée du 15 mars, comme au Cours 06 :

```
docker compose exec airflow-scheduler airflow dags trigger boreal_quotidien --logical-date 2026-03-15T00:00:00+00:00
```

Le mensuel rattrape ses trois mois de lui-même, en une minute et demie. Le quotidien crée d'abord l'exécution de la veille, comme au Cours 06, et fait attendre la vôtre derrière elle, puisqu'il n'en tient qu'une à la fois. Comptez une dizaine de minutes, puis :

```
docker compose exec airflow-scheduler airflow dags list-runs boreal_mensuel
docker compose exec airflow-scheduler airflow dags list-runs boreal_quotidien
```

```
dag_id         | run_id                                | state   | run_after                 | logical_date
===============+=======================================+=========+===========================+==========================
boreal_mensuel | scheduled__2026-04-01T00:00:00+00:00 | success | 2026-04-01T00:00:00+00:00 | 2026-03-01T00:00:00+00:00
boreal_mensuel | scheduled__2026-03-01T00:00:00+00:00 | success | 2026-03-01T00:00:00+00:00 | 2026-02-01T00:00:00+00:00
boreal_mensuel | scheduled__2026-02-01T00:00:00+00:00 | success | 2026-02-01T00:00:00+00:00 | 2026-01-01T00:00:00+00:00

dag_id           | run_id                                    | state   | run_after                         | logical_date
=================+===========================================+=========+===================================+=========================
=
boreal_quotidien | manual__2026-09-23T15:29:09.987325+00:  | success | 2026-09-23T15:29:09.987325+00:00 | 2026-03-
15T00:00:00+00:00
                 | 00                                        |         |                                   |
boreal_quotidien | scheduled__2026-09-23T00:00:00+00:00   | failed  | 2026-09-23T00:00:00+00:00         | 2026-09-
22T00:00:00+00:00
```

Les colonnes des heures de début et de fin sont coupées ici ; les dates des exécutions de la veille et du déclenchement à la main seront celles de votre séance. Les trois mois du mensuel sont au vert : chacun a jugé le contrat de l'ERP sur son mois, et aucun ne porte de défaut. Le 15 mars est au vert aussi. La veille est rouge. Regardez ses tâches, avec le `run_id` que vous donne votre propre liste :

```
docker compose exec airflow-scheduler airflow tasks states-for-dag-run boreal_quotidien scheduled__2026-09-23T00:00:00+00:00
```

```
dag_id           | logical_date              | task_id   | state
=================+===========================+===========+================
boreal_quotidien | 2026-09-22T00:00:00+00:00 | contrats  | failed
boreal_quotidien | 2026-09-22T00:00:00+00:00 | dbt_build | upstream_failed
boreal_quotidien | 2026-09-22T00:00:00+00:00 | bronze    | success
boreal_quotidien | 2026-09-22T00:00:00+00:00 | raw       | success
boreal_quotidien | 2026-09-22T00:00:00+00:00 | corpus    | success
```


© 2026 iAlexMG. Tous droits réservés.

Le bronze, la `RAW` et le corpus ont tourné ; la tâche des contrats a échoué, deux fois à cinq minutes d'écart, comme le DAG le prévoit ; et `dbt_build` n'a jamais été lancée, `upstream_failed`. Ouvrez le journal de la tâche dans l'interface, http://localhost:8080 (`airflow` et `airflow`) : le DAG, l'exécution rouge, la tâche `contrats`, son journal. Les lignes du pipeline y paraissent, chacune précédée de son heure :

```
Running command: ['/usr/bin/bash', '-c', 'cd /projet/01_de && python -m pipeline contrats --livraisons usine,telemetrie --depuis
2026-09-22 --jusqua 2026-09-22']
…
ERROR   pipeline :   ROMPUE     telemetrie.archive_du_jour_livree                          1   (attendu = 0)
…
ERROR   pipeline :   ROMPUE     usine_historien.export_du_jour_livre                       1   (attendu = 0)
…
ERROR   pipeline : pipeline arrêté : 2 règles bloquantes rompues : telemetrie.archive_du_jour_livree,
usine_historien.export_du_jour_livre
Command exited with return code 1
```

Aucune archive de télémétrie ni aucun export de l'historien pour la veille : les sources s'arrêtent au 31 mars 2026. Au Cours 06, cette même exécution passait au vert chaque nuit, en chargeant zéro ligne. Elle échouera désormais chaque nuit, et c'est juste : rien n'arrive plus. Si le rouge vous gêne, suspendez le DAG, `airflow dags pause boreal_quotidien` ; ne coupez pas la règle. La tâche prend 25 secondes dans le conteneur, contre une seconde environ sous Windows : le contrat de l'historien relit tout le bronze à travers le montage de WSL 2, le coût de la traversée vu au Cours 06.

Reste avril, que le mensuel n'a pas le droit de créer : sa borne de fin l'arrête au 31 mars. Pour juger sa tâche sur ce mois quand même, `airflow tasks test` la lance seule, pour la date qu'on lui donne, dans une exécution temporaire qu'il efface ensuite :

```
docker compose exec airflow-scheduler airflow tasks test boreal_mensuel contrats_du_mois 2026-04-01
```

```
… INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
… ERROR   pipeline :   ROMPUE     erp_extraits.extrait_du_mois_livre                      1   (attendu = 0)
… ERROR   pipeline :   ROMPUE     erp_extraits.commandes_du_mois                          0   (attendu >= 300)
…
… Command exited with return code 1
… Task failed with exception
```

La même commande que dans le DAG, `{MOIS}` rendu en `2026-04` par la date qu'on a donnée, et le même verdict qu'à l'étape 3.

Piège — une exécution hors des bornes du DAG. `airflow dags trigger boreal_mensuel --logical-date 2026-04-01T00:00:00+00:00` ne refuse rien : l'exécution naît, passe à `success` en quelques millisecondes, et `airflow tasks states-for-dag-run` répond `No data found`. Airflow 3.3.1 n'y lance aucune tâche, parce que la date est après la borne de fin, et la marque réussie quand même. Il en va de même pour le quotidien déclenché avant sa `start_date`. Un vert sans tâche, c'est exactement le défaut que ce cours chasse : pour juger une date hors des bornes, passez par `airflow tasks test`.

Étape 11 — Le post-mortem du micrologiciel 2.0

Théorie : `Theorie_11_DE.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

L'incident du micrologiciel 2.0 est détecté, qualifié par l'étape 1, contenu et corrigé par l'étape 4. Reste à l'écrire. Mesurez d'abord son impact sur ce qui comptait le plus : les motoneiges de la plage de l'incident de l'usine, `MN26001238` à `MN26001466`, celles que le compresseur 2 a mal serrées. Dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire`, l'entrepôt en lecture seule :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

SELECT COUNT(*) AS messages, COUNT(DISTINCT message->>'sernr') AS vehicules,
       COUNT(message->>'temp_moteur_max') AS ancien_champ,
       COUNT(*) FILTER (WHERE CAST(message->'codes' AS VARCHAR) LIKE '%T310%') AS avec_t310,
       MIN(_lot) FILTER (WHERE CAST(message->'codes' AS VARCHAR) LIKE '%T310%') AS premier_t310
FROM raw.telemetrie_sorties
WHERE message->>'sernr' BETWEEN 'MN26001238' AND 'MN26001466';
```

messages int64	vehicules int64	ancien_champ int64	avec_t310 int64	premier_t310 varchar
1044	93	0	23	2025-12-24

Quatre-vingt-treize motoneiges ont publié 1 044 messages, et aucun ne porte l'ancien nom du champ : elles sont sorties de l'usine avec le micrologiciel 2.0. Vingt-trois de leurs sorties portent le code de surchauffe T310, la première le 24 décembre 2025. Pour elles, un lecteur de `temp_moteur_max` n'a jamais rien vu. Sur toute la flotte, en 2026 :

```
SELECT COUNT(*) AS sorties_t310,
       COUNT(*) FILTER (WHERE CAST(message->>'temp_moteur_max' AS DOUBLE) >= 110)
           AS vues_par_l_ancien_champ,
       COUNT(*) FILTER (WHERE CAST(message->>'temperature_moteur_max' AS DOUBLE) >= 110)
           AS vues_par_le_nouveau
FROM raw.telemetrie_sorties
WHERE _lot >= '2026-01' AND CAST(message->'codes' AS VARCHAR) LIKE '%T310%';
```

sorties_t310 int64	vues_par_l_ancien_champ int64	vues_par_le_nouveau int64
38	0	16

Trente-huit sorties en surchauffe, dont seize à 110 °C ou plus, et zéro pour qui lit l'ancien nom. Les codes d'anomalie, eux, n'ont pas changé de nom : c'est par eux que l'incident de l'usine s'est vu en aval.

© 2026 iAlexMG. Tous droits réservés.

Quittez DuckDB par `.exit`. Écrivez maintenant le post-mortem, dans le dépôt, à côté du code qui a permis de le reconstituer :

```
mkdir postmortems
notepad postmortems\micrologiciel_2_0.md
```

```
# Post-mortem : le champ renommé du micrologiciel 2.0

*Incident de données reconstitué au Cours 11 sur l'historique de la `RAW`, génération de Boréal
jusqu'à 2026-03. Rédigé sans blâme : il cherche ce qui, dans le système, a laissé passer l'erreur.*

| | |
|---|---|
| Début | 2025-05-01, premier message du micrologiciel 2.0 |
| Détection | par le contrat `telemetrie` 1.0.0, rejoué sur l'historique : règle `champs_du_message` rompue le 2025-05-01, quatre
messages hors contrat |
| Gravité | haute : une grandeur de sécurité, la température du moteur, devient illisible pour qui la lit par son ancien nom |
| Produit touché | `raw.telemetrie_sorties`, et tout ce qui lit `message->>'temp_moteur_max'` |
| Correction | le contrat en version 2.0.0, qui déclare les deux formes du message |

## Ce qui s'est passé

Le micrologiciel 2.0 du module télématique, diffusé à partir du 2025-05-01, a renommé
`temp_moteur_max` en `temperature_moteur_max` et ajouté `heures_moteur`. Rien ne l'annonçait au
chemin des données. Le message arrive en JSON et entre tel quel dans la `RAW` : aucune erreur,
aucune ligne perdue, et c'est précisément pourquoi personne ne l'a vu.

| Date | Fait |
|---|---|
| 2025-04-30 | le contrat 1.0.0 est respecté : tous les messages portent les treize champs des micrologiciels 1.x |
| 2025-05-01 | quatre messages sous la forme 2.0 ; la règle de schéma, bloquante, arrête le pipeline |
| mai 2025 | 77,8 % des messages portent encore `temp_moteur_max` |
| juin 2025 | 35,3 % |
| juillet 2025 | 4,2 % ; le dernier message sous l'ancien nom arrive le 2025-07-14 |
| août 2025 | 0 % : un lecteur de l'ancien champ ne voit plus aucune température |
| 2025-12-24 | premier code T310, surchauffe, d'une motoneige de la plage de l'incident de l'usine |
| février 2026 | la température réelle atteint 120,7 °C, dans le champ que l'ancien lecteur ne lit pas |
```

La suite, l'impact, la cause et les actions :

```
## L'impact

| Mesure | Valeur |
|---|---|
```


© 2026 iAlexMG. Tous droits réservés.

```
| Messages du micrologiciel 2.0 au 2026-03-31 | 115 807 |
| Motoneiges de la plage `MN26001238` à `MN26001466` qui ont publié | 93, pour 1 044 messages, tous en 2.0.1 |
| Leurs messages lisibles par l'ancien champ | 0 |
| Sorties portant T310 en 2026 | 38, dont 16 à 110 °C ou plus, lisibles seulement par le nouveau champ |

Sans contrat, une surveillance de la température du moteur écrite en 2024 serait restée muette
pendant tout l'hiver des surchauffes. Les codes d'anomalie, eux, n'ont pas changé de nom : c'est
par eux que l'incident de l'usine s'est vu en aval.

## La cause

Rien n'obligeait l'équipe du module télématique à annoncer un changement de format, et rien ne
vérifiait la forme des messages à l'entrée. Les lecteurs désignaient la température par son nom,
et un nom qui change ne fait pas planter une requête : il la fait rendre `NULL`.

## Ce qui a tenu

- La `RAW` a gardé chaque message entier, en JSON : aucune donnée n'est perdue, tout se relit.
- La table Iceberg du Cours 10 désigne ses colonnes par identifiant : la température s'y lit sous un
  seul nom, pour les deux formes.
- Le journal des chargements et la fenêtre du Cours 06 permettent de rejouer les jours touchés sans
  rien doubler.

## Les actions

| Action | Responsable | Échéance | État |
|---|---|---|---|
| Passer le contrat `telemetrie` en 2.0.0, avec les deux formes du message | Data Engineering | à la détection | fait |
| Relire la température par `COALESCE(temperature_moteur_max, temp_moteur_max)` dans chaque lecteur | Analytics Engineering, Data
Science | avant la prochaine livraison | à faire |
| Annoncer au contrat toute nouvelle version du message avant sa diffusion | l'équipe du module télématique | à la prochaine
version du micrologiciel | à faire |
| Retirer la forme 1.x du contrat, en version 3.0.0, une fois la flotte passée en 2.0 | Data Engineering | quand aucun message
1.x n'arrive plus depuis un mois | à faire |
```

Lisez la cause : elle ne nomme personne. Elle dit ce qui, dans le système, a laissé passer l’erreur : rien n’obligeait à annoncer un changement de format, rien ne le vérifiait. Chaque action a un responsable et une échéance, et celle qui compte le plus pour l’avenir n’appartient pas à Data Engineering : c’est l’équipe du module télématique qui annoncera ses versions au contrat. Le post-mortem vit dans le dépôt, versionné : dans un an, il dira pourquoi le contrat de la télémétrie porte deux formes, et quand la troisième version pourra en retirer une.

Étape 12 — Versionner

© 2026 iAlexMG. Tous droits réservés.

Depuis `01_de` :

```
git add pipeline/contrats.py pipeline/__main__.py rejouer_contrats.py
git add contrats/erp_extraits.yaml contrats/telemetrie.yaml
git add contrats/usine_historien.yaml contrats/argent_usine.yaml
git add dags/boreal_quotidien.py dags/boreal_mensuel.py postmortems/micrologiciel_2_0.md
git commit -m "Cours 11 : les contrats de données, leur rejeu, les DAG et le post-mortem"
```

Les verdicts ne se versionnent pas : `pipeline.controles` vit dans l'entrepôt, hors du dépôt, et se refait à tout moment par `python -m pipeline contrats`. Ce qui se versionne, c'est ce que les contrats promettent, et c'est ce qu'un changement de contrat laisse dans l'historique de Git, avec son auteur et sa raison.

Récapitulatif

Ce que vous avez mesuré	**Le chiffre**
Trois défauts qui passent au vert	avril 2026 « à jour », code 0 ; 609 commandes sous deux lots ; `temp_moteur_max` à 0 % dès août 2025
Le contrat de l'ERP	609 relivrées et 1 concessionnaire en avertissement ; avril 2026, deux règles bloquantes, code 1
La télémétrie	4 messages hors de la 1.0.0 le 2025-05-01, 115 807 sur l'historique ; 0 en 2.0.0
Le lignage	les 609 relivrées dans un seul fichier, `2023-03_bis/VBAK.csv`, de même empreinte que celui de `2023-03`
L'historien	64 points douteux, 529 perdus aux coupures, 1 capteur figé, le 2025-10-08
La pression	15 alertes pour la version de 7 h à 14 h, dont 8 fausses ; 7 pendant la marche des lignes, 0 fausse ; 3 jours manqués
Le rejeu de 182 jours	107 alertes, puis 8 une fois deux règles en suivi
L'argent	4 promesses tenues, 10 heures hors de la plage du capteur
Les 22 règles, sur tout l'historique	3,1 s ; 609, 1, 1, 7 et 10 en avertissement, 64 et 529 en suivi, aucune bloquante ; code 0
Les DAG	les trois mois du mensuel au vert ; le 15 mars au vert ; la veille au rouge, deux règles bloquantes, `dbt_build` jamais lancée ; avril par `tasks test`, code 1

© 2026 iAlexMG. Tous droits réservés.

Ce que vous avez mesuré	Le chiffre
L'incident	93 motoneiges de la plage, 1 044 messages, 0 lisible par l'ancien champ ; 38 sorties T310 en 2026, 16 à 110 °C ou plus

Ce que vous savez maintenant et que vous ne saviez pas : un pipeline qui ne plante pas n'a rien prouvé ; une règle de fraîcheur compare ce qui est là à ce qui était attendu ; bloquer arrête ce qui se construit, pas la donnée ; un contrat a une version, et un changement qui casse se négocie ; une règle de distribution se bâtit sur la vraie sortie, en regardant chaque alerte ; et une alerte se juge par ce qu'elle fait faire à quelqu'un, pas par ce qu'elle a de vrai.

Prochaine étape (Cours 12). Le capstone reprend tout, de bout en bout : les deux chemins, les contrats à chaque livraison, et la plateforme reconstruite de zéro.

Dépannage

Symptôme	Cause	Ce qu'on fait
`ModuleNotFoundError: No module named 'yaml'`	PyYAML n'est pas installé dans `formations`	`pip install -r ..\01_Data_Engineering\requirements.txt`, depuis `Projet_Boreal_intermediaire`
`Étape inconnue : contrats.`	`pipeline\__main__.py` n'a pas sa ligne dans `ETAPES`, ou l'import manque	les trois changements de l'étape 2
`EntrepotOccupe: L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.`	la ligne de commande DuckDB tient l'entrepôt ouvert	`.exit` dans DuckDB, puis relancer
`pipeline arrêté : erp_extraits.yaml : il manque version`	une clé de tête manquante ou mal écrite	comparer l'en-tête du fichier à celui du labo
`pipeline arrêté : while scanning for the next token`, puis `found character '\t' that cannot start any token`	une tabulation dans le fichier YAML	la remplacer par des espaces : le YAML n'accepte qu'eux pour l'indentation
`pipeline arrêté : erp_extraits.yaml, extrait_du_mois_livre : sévérité bloquant, avertissement, suivi`	la clé écrite `sévérité`, avec ses accents, ou une valeur mal orthographiée	les clés et les valeurs du contrat s'écrivent sans accent : `severite: bloquant`
`Conversion Error: Malformed JSON at byte 0 of input` sur la télémétrie	les parenthèses autour de `t.message->>'sernr'` ont sauté : DuckDB lit `(z.SERNR = t.message)->>'sernr'`	remettre `z.SERNR = (t.message->>'sernr')`

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause	Ce qu'on fait
`commande_relivree_a_l_identique` rend 27 746	l'étoile manque devant `COLUMNS` : la règle ne hache que `MANDT`	`*COLUMNS(* EXCLUDE (_fichier, _lot, _charge_le))`
`pipeline arrêté : while scanning a simple key` sur le contrat de l'usine	le SQL de `preparation` n'est pas indenté sous sa clé	deux espaces devant chaque ligne du SQL qui suit la clé `preparation`
`IO Error: No files found that match the pattern "plateforme/lac/bronze/usine/historien/**/*.parquet"`	le bronze de l'historien n'existe pas, ou le pipeline ne travaille pas depuis `Projet_Boreal_intermediaire`	`python -m pipeline bronze`, depuis `01_de`
le rejeu donne 0 tranche	`--depuis` après `--jusqua`, ou une date mal écrite	des dates `AAAA-MM-JJ`, la première avant la seconde
La tâche `contrats` n'apparaît pas dans le DAG	le lecteur de DAG n'a pas encore relu le fichier, ou le fichier ne se lit plus	`airflow dags list-import-errors`, qui nomme la ligne fautive ; sinon, attendre une demi-minute
Le DAG quotidien rouge chaque nuit, sur `archive_du_jour_livree` et `export_du_jour_livre`	les sources s'arrêtent au 31 mars 2026 : la règle de fraîcheur dit vrai	`airflow dags pause boreal_quotidien` ; ne pas couper la règle
Une exécution au vert en quelques millisecondes, sans aucune tâche	une date hors des bornes du DAG, avant sa `start_date` ou après sa `end_date`	`airflow tasks test <dag> <tâche> <date>`
`airflow tasks clear` s'arrête sur `TypeError: unsupported type for timedelta seconds component: dict`	une limite d'Airflow 3.3.1 sur les DAG qui portent une Deadline Alert (Cours 06)	`airflow dags delete <dag_id> -y`, puis attendre qu'il revienne

© 2026 iAlexMG. Tous droits réservés.